

Hybrid MPI+OpenMP Performance of PETSc Krylov Solvers on ARCHER2

Leyan Li

MSc in High Performance Computing
EPCC, University of Edinburgh

August 2026

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science.

Abstract

PETSc can combine MPI ranks with OpenMP threads, but the best balance depends on the solver, decomposition and concurrency. This dissertation compares four full-node layouts on ARCHER2: flat MPI and hybrid layouts with two, four or eight threads per rank. Matched 2D and 3D Poisson benchmarks cover 5.0–164.6 million unknowns on 1–32 nodes. Each formal configuration is run three times, and every run performs one warm-up solve that builds the GAMG hierarchy, followed by 19 timed solves that reuse it. Eleven admitted size–node pairs form two weak-scaling chains at approximately constant work per core.

In 2D, hybrid layouts improve relative to flat MPI as concurrency grows, and four of the six layout–chain series improve at every step. 64×2 is faster per iteration than flat MPI at all 11 pairs and reaches 13.1% higher raw throughput at 164.56 million unknowns on 32 nodes. The 3D advantage is larger, reaching 33.2%, but no series improves at every step and the best layout changes with scale and node count. Eight threads per rank wins no weak-scaling point in either dimension.

A matched extension holds the problem near 20 million unknowns while scaling from one to 32 nodes. All four 2D layouts still improve at 4096 cores. In 3D, flat MPI stops improving and starts getting slower after eight nodes, and the intermediate layouts after 16, while 16×8 is still improving at 32 and is $3.81\times$ faster than flat MPI there. PETSc event data associate the hybrid advantage with lower exposure to halo exchange and global reduction. Eleven Linaro MAP profiles show an opposing rise in OpenMP-runtime waiting as threads replace ranks, while threaded BLAS accounts for at most 2.1% of core time during the solve. These measurements locate the software costs but do not establish the hardware mechanism.

Acknowledgements

Contents

Abstract	i
Acknowledgements	ii
Abbreviations	ix
1 Introduction	1
1.1 Motivation	1
1.2 Aims and Research Questions	1
1.3 Contributions	2
1.4 Dissertation Structure	2
2 Background and Related Work	3
2.1 Parallel Programming Models	3
2.2 Limitations of Flat MPI	4
2.3 Hybrid MPI+OpenMP	4
2.4 ARCHER2 Node Architecture and NUMA	5
2.5 PETSc and Krylov Solvers	6
2.6 Performance Analysis Tools	7
2.7 Summary and Research Gap	7
3 Experimental Platform and Methodology	9
3.1 Hardware and Software Environment	9
3.2 Benchmark Problems and Drivers	9
3.3 Solver Configuration	10
3.4 Configuration Space and Selection Rules	11
3.5 Bounding the Thread Count	12
3.5.1 Exploratory Single-Node Sweep	12
3.5.2 The Same Bound on the Production Configuration	13
3.6 Job Configuration and Thread Placement	14
3.7 Metrics	14
3.8 Superseded Campaigns	15
3.9 Reproducibility	16
4 2D and 3D Problems	17
4.1 Experimental Setup	17
4.1.1 Weak-Scaling Chains	17
4.1.2 Strong-Scaling Design	19
4.2 Weak Scaling at Constant Work per Core	19
4.2.1 2D	19
4.2.2 3D	20
4.2.3 Layout Ranking and Efficiency	21

4.3	Strong Scaling at Fixed Problem Size	22
4.3.1	2D	22
4.3.2	3D	23
4.3.3	Best Layout and the 32-Node Endpoint	25
4.4	Interpretation	25
5	Profiling and Bottleneck Analysis	27
5.1	Why a Repeated-Solve Driver	27
5.2	Analysis Methodology	27
5.3	MAP Profiling Results	28
5.4	Bottleneck Identification	30
5.4.1	What is Measured	30
5.4.2	Halo Exchange Dominates the Exposure, and Hybridisation Reduces It	30
5.4.3	Reduced Exposure is Necessary but not Sufficient	32
5.4.4	What This Does and Does Not Establish	35
6	Discussion	37
6.1	Answering the Research Questions	37
6.1.1	RQ1: Does the threaded backend reduce time to solution, and by how much?	37
6.1.2	RQ2: At which balance, and how does the answer change with the regime?	37
6.1.3	RQ3: Which parts of the solve account for the difference?	38
6.1.4	RQ4: How far do the trends generalise, and what would establish a mechanism?	38
6.2	Threats to Validity	39
7	Conclusion and Future Work	40
7.1	Conclusions	40
7.2	Practical Guidance for PETSc on ARCHER2	41
7.3	Future Work	41
	Bibliography	43
A	Reproducibility: Repository and Scripts	44
A.1	Layout	44
A.2	Evidence Index	45
A.3	Regenerating the results	47
B	Supplementary Figures and Tables	49
B.1	Weak-scaling throughput	49
B.2	Fixed-size runtime grids	50
C	Full log_view Excerpts	52

List of Figures

2.1	Hierarchy of an ARCHER2 compute node. Each node has two 64-core AMD EPYC 7742 processors, eight 16-core NUMA regions, and four-core CCXs that each share 16 MB of L3 cache. Source: ARCHER2 hardware documentation [8]. .	6
3.1	How PETSc's default row blocks shape a subdomain. Contiguous rows of the lexicographic numbering make each rank's subdomain a stack of complete $m \times n$ planes, so the halo is the two planes it shares with its neighbours in k and cannot fall below $2L$ once L drops under mn . A Cartesian decomposition would cut the domain in every direction instead, giving six smaller faces. Every run in this dissertation uses (a); (b) is the alternative Section 7.3 proposes. The shaded block is one rank's subdomain and the dark surfaces are what it exchanges.	10
3.2	Single-node default-solver sweep on a 1000×1000 grid. Adding OpenMP threads to one MPI rank leaves the cost per iteration almost unchanged. Adding MPI ranks at one thread each reduces it by two orders of magnitude. Both lines start from the same one-rank, one-thread run.	13
4.1	2D time per KSP iteration relative to the 128×1 baseline at the same size-node pair. Values below 1 mean the hybrid layout is faster. Whiskers propagate the observed range of three formal runs.	19
4.2	Median 2D time per KSP iteration along the two unknowns/core chains. A horizontal line would be ideal weak scaling.	20
4.3	3D time per KSP iteration relative to the 128×1 baseline at the same size-node pair. Unlike the 2D case in Figure 4.1, no series is monotone.	21
4.4	Median 3D time per KSP iteration along the two unknowns/core chains.	21
4.5	2D time per solve at a fixed 20.25M unknowns, one to 32 nodes. Both axes are logarithmic, so ideal strong scaling is a straight line of slope -1 . Whiskers show the observed range of three formal runs.	23
4.6	3D time per solve at a fixed 20.35M unknowns. Flat MPI bottoms at eight nodes, 64×2 and 32×4 at 16, and only 16×8 still improves at 32.	24
4.7	3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. Because the baselines differ, this figure ranks scaling behaviour and not speed. Flat MPI falls to 10.9% at 32 nodes while 16×8 retains 44.5%. .	24
5.1	Where the cores are during the solve, from the MAP call trees. Within each panel the OpenMP waiting share grows and the MPI share shrinks as threads replace ranks, and both effects strengthen with node count. Paired bars are the two PETSc builds at the same configuration.	29
5.2	2D share of <code>RepeatedSolves</code> KSPSolve time spent inside halo exchange and global reduction events, panelled by unknowns/core chain as in Chapter 4. Only the 40k chain reaches 32 nodes. Each point is the median of three repetitions. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.	31

5.3	3D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events, panelled and styled as in Figure 5.2. Note the vertical scale: flat MPI reaches 84.6% at the largest point.	31
5.4	3D exposure removed relative to flat MPI against the resulting time per iteration. Colour is the layout and point labels are node counts. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates. The two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim 40k$ unknowns/core and dash-dotted with hollow markers for $\sim 80k$. In 3D the three stay together, every layout's per-chain break-evens falling within 2.4 points of its pooled value. Each line is clipped to the range it was fitted to, so none is extrapolated. The fits are descriptive, not predictive.	34
5.5	2D exposure removed relative to flat MPI against the resulting time per iteration, styled as in Figure 5.4. Here the solid pooled fits are not summaries of their parts. For 16×8 the solid line lies flat while its two chains, correlating at -0.91 and -0.96 separately, descend steeply across it. 64×2 and 32×4 depart from their pooled fits as well. Note also that 16×8 lies above parity at every one of its 11 points, which is read off the markers and not off any fit.	35
B.1	Median 2D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs. All layouts use 128 cores per node, so a fixed node count means a fixed core count.	49
B.2	Median 3D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs.	50

List of Tables

2.1	The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation. It inherits both cost columns.	4
2.2	Four fully populated layouts under close binding. All use 128 physical cores. What changes is the rank count and where a rank sits relative to the shared-L3 boundaries. The table describes placement, not its performance effect.	5
2.3	PETSc events and measured calls in one GAMG-preconditioned CG step. A step is one preconditioner application. CG performs one more step per solve than its reported iteration count, because it preconditions the initial residual.	7
3.1	Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging.	11
3.2	Cost per KSP iteration for the eight fully populated single-node layouts (ranks $\times$ threads = 128) on a 1000×1000 grid. The trend is monotone in the rank count, with no threshold at any particular thread count. The 128×1 figure is measured over its non-converged 10,000 iterations, which still gives a valid per-iteration cost.	13
3.3	KSP time per iteration for five full-node layouts in the production configuration. Each cell has one run and every row rises with threads per rank. The campaign has no 128×1 cell and bounds the search space rather than ranking the admitted layouts.	14
3.4	Relative tolerance actually used by ex2 on a 100×100 grid, read from -ksp_view , for the three option styles. The double dash is silently ignored and the mesh-dependent default survives. -options_left names the unconsumed option explicitly.	15
4.1	Grids and node counts. Weak-scaling points pass the unknowns-per-core filter of Section 3.4. The fixed-size campaign runs the 20m problem over all nodes.	17
4.2	The 11 admitted pairs arranged as two weak-scaling chains with near-constant unknowns per core.	18
4.3	Fastest layout by median throughput over three repeats, its margin over flat MPI, and time per iteration. The 2D 41m/8-node row is a tie because ranges overlap.	22
4.4	Weak-scaling efficiency (%) at the largest node count of each chain, relative to the same layout's one-node time. Each layout is measured against itself, so this quantifies degradation, not absolute speed.	22
4.5	Fastest layout for each fixed 20M problem, by three-run median time per solve, and its advantage over flat MPI.	25
4.6	The 32-node endpoint of the fixed-size curves. Speedup and cumulative efficiency are measured against the same layout's one-node time, so they reward a poor baseline. The doubling efficiency is the 16-to-32-node step alone. Time per solve is the only column that compares layouts directly.	25

4.7	Cross-dimensional weak- and strong-scaling summary. Throughput gain and time saving use rate and time conventions respectively. A layout turns upward when its time after its measured minimum increases.	26
5.1	Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8	27
5.2	Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. OpenMP is the GNU runtime's spin and wait paths, that is threads with no work. MAP instrumentation inflates these runs, so the ordering between layouts is the result; the absolute values do not transfer to the uninstrumented campaigns. . .	28
5.3	The exposure share of Figure 5.2 and Figure 5.3 split into its two event classes, at four representative points along the $\sim 40k$ unknowns/core chain of Table 4.2, for the 128×1 and 16×8 layouts. Halo is VecScatterBegin and VecScatterEnd ; reduction is VecTDot and VecNorm . Their sum is the quantity plotted, so it is left to the figures. Each row is the repetition whose total is the median of its three, so both components come from the same run. The two are rounded independently and at one decimal need not sum to the plotted total (3D, 32 nodes, 128×1 : 70.9 and 13.6 against a plotted 84.6). Percentage points of RepeatedSolves KSPSolve time.	32
5.4	ExposureRemoved against TimeRatio, both relative to flat MPI at the same size-node point, the first in percentage points. The break-even value is the exposure reduction the fitted line requires to offset threading overhead. a and b are that line's slope and intercept, tabulated as $10^3 a$, so $(1 - b)/a$ reproduces the last column. <i>None</i> marks $a \geq 0$, where the line never descends to parity and no break-even exists. Eleven points per layout. The fits are descriptive only.	33
A.1	Repository layout. Raw logs and retrieved job CSVs are immutable evidence. Everything under a <i>derived</i> heading is regenerated by the scripts.	44
A.2	Evidence index. Script paths are relative to analysis/scripts/ , derived and raw paths to analysis/ . Log counts are the files present, which for the 3D fixed-size campaign exceeds the 72 accepted runs because superseded job attempts are retained. The last two rows are read directly from a named file and have no derived stage.	46
B.1	2D fixed-size campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row. All layouts fill the node, so 32 nodes is 4096 cores in every row. Iteration counts range from 9 to 11.	50
B.2	3D fixed-size campaign, 273^3 and 20.35M unknowns, median seconds per solve over three runs at every layout and node count. Columns as in Table B.1. A 16-to-32-node figure below 1.00 means the layout is slower at 32 nodes than at 16. Iteration counts are 6 or 7 throughout.	51

Abbreviations

AMG	Algebraic multigrid
BLAS	Basic Linear Algebra Subprograms, the standard interface for dense vector and matrix operations
CCD	Core Complex Die, two CCXs on an AMD Zen 2 processor
CCX	Core Complex, four cores sharing one 16 MB L3 cache
CG	Conjugate gradients
GAMG	PETSc's algebraic multigrid preconditioner (PCGAMG)
GMRES	Generalised minimal residual method, PETSc's default Krylov solver
ICC	Incomplete Cholesky factorisation
KSP	PETSc's Krylov subspace solver object
LAPACK	Linear Algebra PACKage, the dense factorisation layer above BLAS
LibSci	HPE Cray Scientific Libraries, the vendor's tuned BLAS and LAPACK
MAP	Linaro MAP, the sampling profiler used in Chapter 5
MPI	Message Passing Interface
NUMA	Non-uniform memory access
OpenMP	Open Multi-Processing, the shared-memory threading interface
PC	PETSc's preconditioner object
PETSc	Portable, Extensible Toolkit for Scientific Computation
SMT	Simultaneous multithreading

Chapter 1

Introduction

1.1 Motivation

Sparse iterative solvers dominate many scientific applications, and PETSc [1, 2] is widely used to implement them. Since version 3.22, PETSc can supplement its distributed-memory MPI model with threaded kernels [3]. A fixed core allocation can therefore use flat MPI or divide each process across OpenMP threads.

The choice has costs on both sides. An ARCHER2 compute node provides 128 physical cores across two sockets, eight NUMA regions, and thirty-two four-core shared-L3 domains. Flat MPI makes every subdomain a separate process, with its own address space, its own message matching, and its own copy of the solver’s state. Replacing ranks with threads reduces the number of MPI endpoints, the processes that send and receive messages, and with them the number of subdomain boundaries. It also adds the cost of creating and synchronising thread teams, and makes the threads inside a rank share caches and memory channels.

Core count alone does not determine the balance. The possible saving depends on how much of an iteration is exposed to communication, which changes with problem size, stencil, preconditioner and node count. The cost depends on how much work each threaded region contains and on where the threads sit in the memory hierarchy. A user allocating a full ARCHER2 node therefore has several plausible layouts but no result that selects one in advance. This dissertation measures that trade rather than assuming either side dominates.

1.2 Aims and Research Questions

This study compares flat MPI with three hybrid layouts for matched 2D and 3D Poisson problems. Six problem sizes cover 5.0–164.6 million unknowns on 1–32 fully populated nodes, with core count, solver and convergence criteria fixed at each comparison. The unknowns-per-core filter leaves each size at only one or two node counts, so those points support weak-scaling chains but not a curve long enough to show where adding nodes stops helping. A matched extension therefore holds about 20 million unknowns fixed across the full node range. Together the campaigns address four questions.

RQ1 Does PETSc’s threaded-kernel backend reduce the time to solution relative to flat MPI on a fully populated node, and by how much?

RQ2 If it does, at which rank–thread balances, and how does the answer change with problem size, node count, dimensionality, and scaling regime?

RQ3 Which parts of the solve account for the difference?

RQ4 How far do the observed trends generalise, and what would be needed to establish a mechanism for them?

Chapter 4 answers RQ1 and RQ2, Chapter 5 answers RQ3, and Chapter 6 addresses RQ4. The feasibility study expected a benefit at two to four threads per rank and identified `VecMDot` and `PCApply` as candidate limits. The later change from GMRES to CG replaced `VecMDot` with `VecTDot`. Section 3.3 explains why.

1.3 Contributions

1. **A matched full-node rank–thread study.** The validated 2D and 3D campaigns cover four layouts, six problem sizes, 1–32 nodes and three formal repeats, followed by matched fixed-size extensions. Each log is checked against the build, process and thread counts, convergence, repeated-solve stage and linked library that it reports, rather than trusted from its filename.
2. **Evidence that concurrency changes the layout ranking.** In 2D the hybrid advantage generally grows along the weak-scaling chains, whereas the 3D preference changes with scale and node count. Small-node comparisons do not transfer directly to larger allocations.
3. **Evidence that the scaling regime changes the recommendation.** Eight threads per rank wins no weak-scaling point, yet 16×8 is the fastest 3D layout when the fixed problem reaches 4096 cores, because it keeps improving to a higher node count than the others.
4. **Event-level evidence for both sides of the trade.** PETSc event data link hybrid layouts to lower halo and reduction exposure at every one of the 22 size–node points, while eleven MAP profiles locate the opposing sampled cost in OpenMP-runtime waiting. The association is descriptive: it identifies software events but does not establish whether cache, memory bandwidth, load imbalance or region length causes the waiting.
5. **A repeated-solve measurement method.** The drivers pay for GAMG setup once and time 19 reused solves in a dedicated PETSc logging stage, so that neither hierarchy construction nor the memory placement the first solve pays for, when a page is assigned to the memory of whichever NUMA region touches it first, dominates the layout comparison.
6. **A reproducible and auditable pipeline.** Raw logs, analysis scripts and figure generation are retained together. The pipeline parses MAP files directly, validates campaign completeness and rejects invalid launcher, solver and tolerance comparisons. Assertions tie tabulated properties to the prose, and Section 6.2 records what the retained evidence cannot establish.

1.4 Dissertation Structure

Chapter 2 covers the execution models, ARCHER2 topology, CG+GAMG solver and profiling tools, then states the research gap. Chapter 3 defines the benchmark drivers, admissible configurations, placement, metrics and validation contract. Chapter 4 presents weak scaling at constant work per core and fixed-size strong scaling without pooling their evidence. Chapter 5 analyses PETSc events and MAP call trees. Chapter 6 answers the research questions and states the threats to validity. Chapter 7 summarises the findings, gives practical guidance, and lists the experiments still required.

Chapter 2

Background and Related Work

2.1 Parallel Programming Models

PETSc applications run on machines whose memory is distributed across nodes and non-uniform within each node. The programming model decides which of these boundaries the program has to handle itself and which the hardware hides from it.

MPI uses independent processes with private address spaces. Processes exchange data through point-to-point operations and collectives defined by the MPI standard [4]. This model matches a distributed-memory machine: each rank owns part of the problem, and communication makes remote dependencies explicit. MPI implementations also run within a node, often through shared-memory transports, so “MPI-only” does not mean that every message crosses the network. It does keep one subdomain per process, the cost of matching each message to its receiver, and a separate copy of the library’s state in every rank.

OpenMP uses a shared-memory, fork-join execution model. Threads in a team can access the same address space, while worksharing constructs divide loop iterations among them [5]. OpenMP removes the need to send messages inside a process, but the runtime still has to create or wake the threads, hand out the work, and make them wait for each other at the end. Threads also compete for cache capacity and memory bandwidth. A threaded loop is therefore faster only when it holds enough work to pay for these costs.

A hybrid program assigns one or more MPI ranks to each node and creates an OpenMP team inside every rank. MPI still communicates between subdomains. OpenMP processes the data owned by each rank. The rank-thread product gives the number of cores used, but two layouts with the same product are not equivalent. For example, 128×1 creates 128 subdomains and MPI endpoints on an ARCHER2 node, whereas 32×4 creates 32 larger subdomains and four threads per subdomain. The second arrangement carries less per-rank solver state and fewer subdomain boundaries to exchange across, but pays for thread scheduling and for the contention between threads sharing a cache and a memory channel. Table 2.1 summarises what each model makes explicit and what it therefore costs.

PETSc uses MPI for distributed-memory communication, so this study compares flat MPI with PETSc’s hybrid path. Other families of programming model are outside its scope.

Table 2.1: The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation. It inherits both cost columns.

Model	Memory	Data exchange	Principal cost
MPI	Private per rank	Explicit messages	Message matching, per-rank solver state, one subdomain boundary per rank
OpenMP	Shared within a team	Shared addresses	Team creation, synchronisation, cache and bandwidth contention
Hybrid	Shared inside a rank, private between ranks	Messages between ranks only	Both, traded against each other by the rank-thread balance

2.2 Limitations of Flat MPI

Flat MPI assigns one rank to each core. Its advantage is uniformity: the same domain decomposition and communication model applies within and between nodes, and each rank executes single-threaded code, which gives predictable placement and avoids thread-safety and OpenMP scheduling costs. A hybrid configuration must outperform this measured baseline, not an abstract MPI implementation.

Strong scaling fixes the global problem while increasing the core count, so work per core falls; weak scaling grows the problem to hold work per core constant, with constant time per iteration as the ideal. Under strong scaling the latency-sensitive operations do not shrink with the local work, because a stencil matrix-vector product still exchanges boundary values and Krylov dot products still require collective reductions, while more of each shrinking subdomain is boundary and less of it is interior. Communication therefore takes a larger share of an iteration. Weak scaling does not remove that cost; it makes it easier to see on its own, because the local kernel work stays steady and any growth in time per iteration comes from what changes with concurrency: neighbour exchange, the number of ranks taking part in a global reduction, and the depth of the reduction tree.

Whether replacing ranks with threads helps under either regime is application-specific. On ARCHER2, Judge found benefits at two or four threads per rank for communication-heavy CP2K and CASTEP runs, but little benefit for the tested molecular-dynamics cases before their scaling limits [6].

2.3 Hybrid MPI+OpenMP

Hybrid performance depends on rank count, threads per rank and placement together. At a fixed core count, adding threads per rank means fewer MPI endpoints and a larger subdomain inside each rank, but every OpenMP region then has more threads to coordinate. Sparse linear algebra is a hard case for this, because it jumps around memory and does little arithmetic per value fetched, so it can saturate memory bandwidth before all the cores are busy.

Lange et al. extended PETSc sparse matrix-vector strong scaling beyond a pure-MPI limit using task-based OpenMP, communication overlap and thread load balancing [7]. Their kernel was written for the purpose, so it took much more than changing the thread count, and its results do not predict how the current backend performs.

PETSc 3.22 added the configure option `-with-openmp-kernels` [3]. This option enables OpenMP in kernels that PETSc has annotated for threaded execution. It does not replace PETSc’s MPI decomposition or make every solver phase threaded. The build studied here uses both `-with-openmp=1` and `-with-openmp-kernels=1`. A run with one thread per rank uses

the same binary as a hybrid run, which isolates the runtime layout from compiler and library differences.

When a hybrid run scales badly, the cause can be too little work per thread, a phase that is serial or dominated by MPI, poor placement, cache effects, or contention for shared memory. Any of these can produce the same ordering of runtimes. Section 2.6 therefore introduces profiling evidence, so that the cost can be located before its cause is argued about.

2.4 ARCHER2 Node Architecture and NUMA

An ARCHER2 compute node contains two 64-core AMD EPYC 7742 (Rome) processors, giving 128 physical cores. The service configures four NUMA regions per socket, eight per node, with 16 cores and local memory channels in each region. A core has private L1 and L2 caches. Four cores form a Core Complex (CCX) and share 16 MB of L3, and two CCXs form a Core Complex Die (CCD). Different CCXs do not share L3 (Figure 2.1) [8].

A diagnostic job confirmed this topology on a compute node. `lscpu` reports two 64-core sockets, 256 logical CPUs, eight NUMA nodes and 512 MiB of L3 in 32 instances. `numactl` assigns 16 physical cores and roughly 32 GB to each NUMA region, with distances 10 within a region, 12 within a socket and 32 across sockets. Both `sysfs` and `hwloc-ls` place one 16 MB L3 above four physical cores. The measured boundaries are therefore four cores per L3 and 16 cores per NUMA region.

NUMA affects both latency and bandwidth. Linux normally places a page of memory in the NUMA region of whichever thread writes to it first. If one thread initialises a large array and a whole team reads it later from several regions, all that traffic falls on the memory controllers of the one region that owns the pages. The ARCHER2 tuning guide also notes that two threads can already come close to the memory-bandwidth limit of a NUMA region for bandwidth-intensive code [9]. More threads do not guarantee more bandwidth.

The cache hierarchy draws a finer boundary than NUMA does: four adjacent cores share an L3 cache, and a 16-core NUMA region holds four such groups. Under the close binding of Section 3.6, the four full-node layouts therefore sit differently against that boundary, as Table 2.2 sets out.

Table 2.2: Four fully populated layouts under close binding. All use 128 physical cores. What changes is the rank count and where a rank sits relative to the shared-L3 boundaries. The table describes placement, not its performance effect.

Layout	Cores/rank	Subdomains/node	Rank against the shared-L3 (CCX) boundary
128×1	1	128	four ranks share one CCX
64×2	2	64	two ranks share one CCX
32×4	4	32	exactly one CCX
16×8	8	16	spans two CCXs

Eight threads span two four-core L3 groups but stay inside one 16-core NUMA region, and no layout tested here puts a rank across two NUMA regions. So the layouts cannot differ because one of them reaches across a NUMA boundary, since none of them does. That rules out one explanation. It does not prove an L3 or OpenMP one. Section 5.3 later shows where the sampled threads wait, without establishing why.

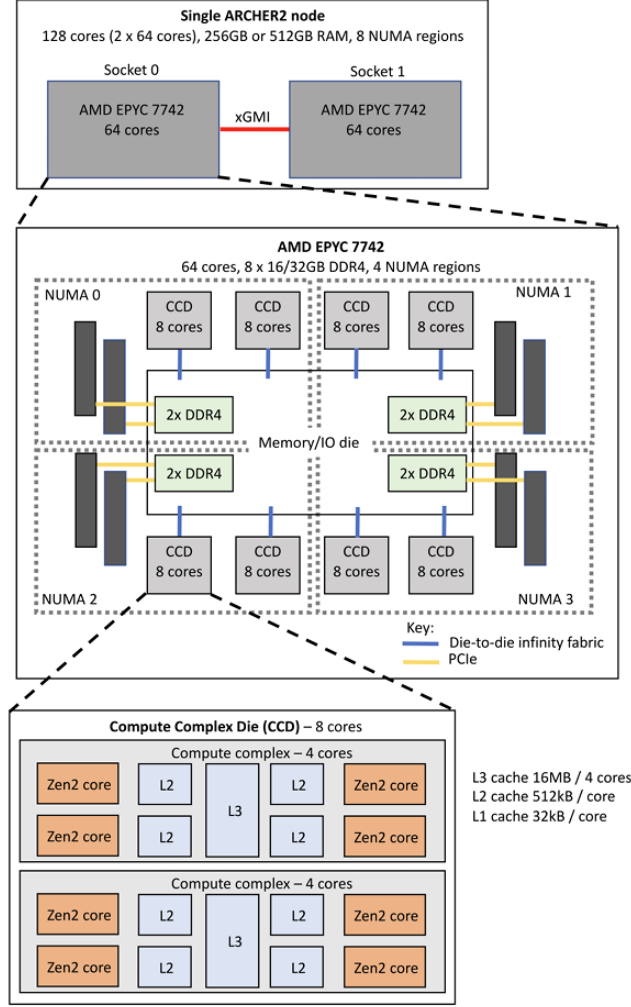


Figure 2.1: Hierarchy of an ARCHER2 compute node. Each node has two 64-core AMD EPYC 7742 processors, eight 16-core NUMA regions, and four-core CCXs that each share 16 MB of L3 cache. Source: ARCHER2 hardware documentation [8].

2.5 PETSc and Krylov Solvers

PETSc provides distributed vectors and matrices, Krylov subspace solvers (KSP), preconditioners (PC), and the logging infrastructure used in this study. PETSc uses MPI for its distributed-memory communication and lets users select solver components at run time [1, 2]. Being able to swap components at run time is useful for experiments. It also means that a comparison of layouts says nothing unless the numerical method is doing comparable work in every layout.

The production system is the finite-difference Laplacian, whose matrix is sparse, symmetric, and positive definite. Conjugate gradients (CG) is designed for this class of linear system [10]. At each iteration CG applies the matrix and the preconditioner, updates vectors, and evaluates inner products. These operations expose two different scaling limits, and the distinction runs through the whole dissertation: the sparse matrix-vector product exchanges halo values with *neighbouring* subdomains, whereas inner products and residual norms require a reduction across *all* ranks. Table 2.3 classifies the work of one CG step accordingly.

The multigrid hierarchy exchanges halos on every level, giving about 43 neighbour exchanges against three global reductions per step. Section 5.4 measures both because they scale differently with rank count.

Table 2.3: PETSc events and measured calls in one GAMG-preconditioned CG step. A step is one preconditioner application. CG performs one more step per solve than its reported iteration count, because it preconditions the initial residual.

Operation	PETSc event	Communication	Calls/step
Matrix-vector product	<code>MatMult</code>	neighbour	28.9
Multigrid restriction	<code>MatMultTranspose</code>	neighbour	7.0
Multigrid prolongation	<code>MatMultAdd</code>	neighbour	7.0
Level residual	<code>MatResidual</code>	neighbour	7.0
Halo exchange, all of the above	<code>VecScatterBegin</code>	neighbour	42.9
Inner products	<code>VecTDot</code>	global	2.0
Residual norm	<code>VecNorm</code>	global	1.0
Vector updates	<code>VecAXPY</code> , <code>VecAYPX</code>	none	44.7

A preconditioner reshapes the system so that CG reaches the requested tolerance in fewer iterations. Algebraic multigrid (AMG) builds a hierarchy of coarser problems directly from the matrix. On each level, smoothing removes the rapidly varying part of the error, and restriction, coarse-grid correction and prolongation deal with the smooth part, which a local smoother would take many sweeps to remove [11]. PETSc’s GAMG (geometric-algebraic multigrid, `PCGAMG`) supplies this hierarchy without requiring a geometric mesh description [1]. The name is historical. No geometric information is given to it here.

GAMG has a setup phase and an application phase. `PCSetUp` builds the aggregates, transfer operators, and coarse matrices. Each subsequent `PCApply` traverses the multigrid hierarchy during `KSPSolve`. A solver that reuses the hierarchy across several systems with the same operator pays the setup once, so first-solve time and repeated-solve time answer different performance questions.

The choice of preconditioner also decides whether the comparison is valid at all. Block Jacobi builds one block per MPI rank, so changing the layout changes the preconditioner itself, and the runs being compared are no longer solving the problem the same way. AMG does not have that property. Section 3.3 measures the effect and settles the comparison on GAMG.

2.6 Performance Analysis Tools

PETSc’s `-log_view` reports time, flop counts, messages, reductions and named events for each logging stage [1]. Events such as `MatMult`, `VecScatterEnd`, `VecTDot`, `PCSetUp` and `PCApply` connect solver operations to measured costs. The report covers every layout, including the 128×1 baseline, and supplies the comparable quantities used across layouts.

Linaro MAP samples source-level call paths together with CPU, memory, vectorisation and MPI metrics [12]. It can tell apart cores running PETSc, cores waiting in MPI, and cores sitting inside the OpenMP runtime. Its instrumentation adds a large and uneven overhead, so MAP is used only to interpret the profiled hybrid layouts, while `-log_view` supplies the timings compared across layouts. Profiled timings are never mixed with uninstrumented scaling results.

2.7 Summary and Research Gap

This chapter has set out the two execution models and the trade between them (Sections 2.1 and 2.3), the ARCHER2 node boundaries that constrain any layout (Section 2.4), the CG+GAMG solver whose halo exchanges and global reductions are the quantities at stake (Section 2.5), and the two instruments that measure them (Section 2.6).

Prior work reports hybrid benefits on ARCHER2 for communication-heavy applications [6] and, for PETSc, for a sparse matrix-vector kernel rewritten for the purpose [7]. Neither covers the threaded-kernel backend that PETSc 3.22 added as an ordinary configure option [3], built here at 3.24.4: the reviewed literature does not evaluate it with CG+GAMG across matched 2D and 3D stencil problems, nor report how its best rank–thread balance moves between the weak- and strong-scaling regimes. The open question is therefore narrower than whether hybrid programming can be fast. It is whether an unmodified PETSc build reduces repeated-solve time when every layout occupies all 128 cores of a node, which balance is best as the job grows, and which measured events accompany the difference. This study holds the core count, solver and convergence criteria fixed at each comparison while varying layout, problem size and node count, then reads the result against `-log_view` and MAP evidence.

Chapter 3

Experimental Platform and Methodology

3.1 Hardware and Software Environment

The node hierarchy relevant to placement is described in Section 2.4. All experiments used physical cores with simultaneous multithreading disabled. The software stack was PrgEnv-gnu 8.4.0, gcc 11.2.0, cray-mpich 8.1.27, and PETSc 3.24.4. The early single-node exploration used `arch-omp-opt`. The validated repeated-solve campaigns used `arch-omp-libsci-rome-opt`, configured with the threaded `libsci_gnu_mp.so.5.0`. LibSci is HPE Cray’s tuned implementation of the Basic Linear Algebra Subprograms (BLAS) and LAPACK, the standard interfaces for dense vector–vector, matrix–vector and matrix–matrix operations. The `_mp` variant threads those kernels with OpenMP. A sparse Krylov solve runs on PETSc’s own kernels and reaches dense BLAS only incidentally. The library is therefore a possible confound rather than a source of speed: its threads would be drawn from the same per-rank thread count and pinned to the same cores as PETSc’s, and would appear in the same OpenMP runtime that Section 5.3 measures. That section reports how much of the solve it accounts for. Both builds enabled PETSc’s OpenMP kernels and used `-O3`. One thread per rank in the OpenMP-enabled build is the flat-MPI baseline.

3.2 Benchmark Problems and Drivers

Both benchmarks solve the Poisson equation $-\nabla^2 u = f$ on a rectangular domain with homogeneous Dirichlet boundary conditions, discretised by second-order central finite differences on a uniform grid. The unknowns are the interior grid points, so a row at the edge of the grid omits the neighbours that fall outside it, and the mesh spacing is absorbed into the right-hand side.

The 2D benchmark is based on PETSc’s KSP tutorial `ex2` (`src/ksp/ksp/tutorials/ex2.c` in the PETSc source tree [13]). On an $m \times n$ grid it assembles the five-point stencil, diagonal 4 and off-diagonals -1 , into an $mn \times mn$ sparse matrix with five nonzeros per row. The 3D benchmark, `3d_repeat`, is a driver written for this project. It assembles the seven-point stencil on an $m \times n \times p$ grid, with seven nonzeros per row and otherwise the same structure. Both matrices are symmetric positive definite, and both drivers set `MAT_SYMMETRIC`.

It was added because the fraction of a subdomain’s rows that couple off-rank differs between the two dimensionalities, and the decomposition is what decides by how much. Both drivers number the grid lexicographically, $Ii = i + jm + kmn$, and both give `MatSetSizes` `PETSC_DECIDE` for the local sizes. PETSc hands each rank a contiguous block of rows, so a subdomain is a slab of that numbering and not a cube (Figure 3.1). Write L for the rows a rank owns. A row couples off-rank through ± 1 , $\pm m$ and, in 3D, $\pm mn$, and the ± 1 term crosses a rank boundary for at

most two rows, so the halo a rank exchanges is about

$$2 \min(L, m) \quad \text{in 2D}, \quad 2 \min(L, mn) + 2 \min(L, m) \quad \text{in 3D}.$$

In 2D the $\pm m$ term is capped at $2m$, that is $2\sqrt{N}$, and stays small beside L . In 3D it is capped at $2mn$, and once L falls below mn every row has both its k neighbours off-rank, so the halo reaches $2L$ and cannot improve further. The 3D problem is therefore not a harder stencil so much as a decomposition that runs out of interior much sooner. Section 6.2 sets out what this scopes the results to.

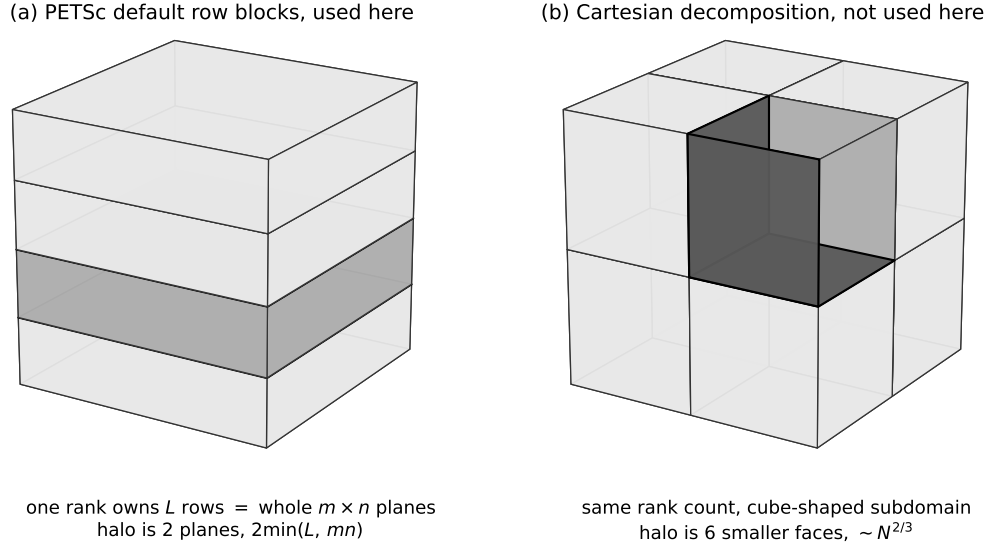


Figure 3.1: How PETSc’s default row blocks shape a subdomain. Contiguous rows of the lexicographic numbering make each rank’s subdomain a stack of complete $m \times n$ planes, so the halo is the two planes it shares with its neighbours in k and cannot fall below $2L$ once L drops under mn . A Cartesian decomposition would cut the domain in every direction instead, giving six smaller faces. Every run in this dissertation uses (a); (b) is the alternative Section 7.3 proposes. The shaded block is one rank’s subdomain and the dark surfaces are what it exchanges.

Neither problem uses an analytic forcing term. Both set the exact solution to $u = \mathbf{1}$ and form $b = Au$, so every run reports $\|x - u\|_2$ as an end-to-end known-solution check that makes a silently wrong solve visible in the log. The grids are matched in unknowns rather than in grid extent: six sizes from 5.0M to 164.6M in each dimension, each 2D size paired with a 3D one to within 0.5%. Table 4.1 lists them with the node counts at which each was run.

Both benchmarks are run through a repeated-solve driver rather than solved once. `ex2_repeat` separates one warm-up `KSPSolve` from 19 timed solves inside a dedicated `RepeatedSolves` logging stage, with the operator and preconditioner unchanged across the timed solves, and `3d_repeat` applies the same design in 3D. The initial guess is zeroed before each timed solve, or the previous solution would already satisfy the tolerance and the next solve would return at iteration zero. Section 5.1 gives the reason and the measured cost split that motivates it. Everything timed in Chapter 4 is a steady-state solve in that sense.

3.3 Solver Configuration

The default `ex2` solver is GMRES with one block-Jacobi block per MPI rank and an ICC factorisation inside each block. A serial run therefore factors the whole matrix, whereas every additional rank changes the preconditioner. At 2240×2240 and above, every run reached PETSc’s 10,000-iteration limit with `DIVERGED_ITS`. Final error norms were of order 10^0 – 10^3 . The jobs

also used an unintended mesh-dependent tolerance, as Section 3.8 establishes, so the observed non-convergence is a joint effect and does not isolate either cause.

Production runs use

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-5 -ksp_atol 1e-10
```

CG applies because the discretised Laplacian is symmetric positive definite. GAMG is used because multigrid convergence is close to mesh-independent for elliptic problems. Across 5.0M to 164.6M unknowns, the final solve takes 9–13 iterations in 2D and 6–7 in 3D. Observed error norms range from 8.5×10^{-3} to 1.34×10^{-1} in 2D and from 6.0×10^{-3} to 8.59×10^{-2} in 3D. Small layout-dependent changes in iteration count affect close throughput comparisons, so the results report both equations solved per second and time per KSP iteration.

Block Jacobi also makes the preconditioner a function of the rank decomposition. Table 3.1 shows the consequence: iteration count is constant across threads at a fixed rank count but varies by more than a factor of three across rank counts.

Table 3.1: Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging.

MPI ranks	OpenMP threads per rank							
	1	2	4	8	16	32	64	128
1	6700	6700	6700	6700	6700	6700	6700	6700
2	3715	3715	3715	3715	3715	3715	3715	
4	2825	2825	2825	2825	2825	2825		
8	4085	4085	4085	4085	4085			
16	8179	8179	8179	8179				
32	6460	6460	6460					
64	7181	7181						
128	10000							

The series repeats exactly across repetitions and is non-monotonic in rank count. Reading across a row changes the thread count but not convergence. Reading down a column changes the block decomposition and the iteration count by more than a factor of three. Layouts with different rank counts therefore apply different preconditioners to the same system, so their wall-clock times cannot answer which rank–thread layout is fastest.

GAMG removes the direct one-block-per-rank coupling. At a fixed size and node count, layouts differ by at most two iterations in 2D and one in 3D, rather than by thousands. Reporting time per outer KSP iteration takes that convergence variation out, although it does not make GAMG hierarchy work or communication identical across decompositions. Results therefore report both raw throughput and time per iteration.

GAMG’s two phases carry very different costs, so the production campaign times repeated solves with the hierarchy reused rather than first solves. Section 5.1 gives the measured split and the driver that produces it. A fuller default-versus-GAMG table would require additional matched converged points and is not used for the layout claims in this dissertation.

3.4 Configuration Space and Selection Rules

Three rules define the formal configuration space. Threads per rank are restricted to $\{1, 2, 4, 8\}$ because 16 or more cost substantially more per iteration in both the exploratory and production configurations (Section 3.5). This is an empirical search bound, not a NUMA or cache claim.

Nodes are fully populated: $\text{ppn} \times \text{threads} = 128$, giving four layouts 128×1 , 64×2 , 32×4 , and 16×8 . Same node count means same core count, so a comparison isolates rank–thread balance from core count.

A configuration at size N on C physical cores is included when $25,000 \leq N/C \leq 100,000$. The completed 2D campaign uses six grids: 2240^2 , 3160^2 , 4500^2 , 6400^2 , 9073^2 , and 12828^2 , or 5.02M to 164.56M unknowns. The filter admits 11 size–node pairs over 1–32 nodes. Each admitted pair runs all four layouts and three formal repeats.

3.5 Bounding the Thread Count

The first rule of Section 3.4 is the only one that had to be measured rather than chosen, and it rests on two campaigns with complementary weaknesses. The exploratory sweep below covers every rank–thread combination on one node, but runs the default solver, whose preconditioner changes with the rank count. The production campaign that follows it runs exactly the solver, build and driver of Chapter 4 at two, four and eight nodes, but only one repeat per configuration. Both give the same answer, and the restriction is applied only because they agree.

3.5.1 Exploratory Single-Node Sweep

The sweep ran every rank–thread combination with $\text{ranks} \times \text{threads} \leq 128$ on a 1000×1000 grid (one million unknowns). It predates the solver change, so it uses the default configuration and, by the argument of Section 3.3, it cannot rank layouts with different rank counts. It can still bound the configuration space, because the conclusions drawn from it hold at a fixed rank count, where the preconditioner is also fixed. Every run reported here comes from the same build and the same campaign, and the two extremes below share one baseline: a single rank on a single thread, 245.7s at 6700 iterations.

Figure 3.2 contrasts the two ways of using the node’s 128 cores, both measured from that shared baseline. Both lines are normalised per KSP iteration, which is what makes them comparable: the threads-only line holds the preconditioner fixed by construction, while the ranks-only line changes it at every point, and its 128-rank configuration never converged.

Pure OpenMP does not scale. A single rank takes 6700 iterations at every thread count, so these runtimes are directly comparable, and they barely move: 245.7s at one thread, 225.5 at eight, 216.5 at thirty-two, 195.6 at sixty-four, and 208.4 at 128. The best case is a factor of 1.26 at 64 threads, after which more threads cost time again. At 128 threads the return is a factor of 1.18 for 128 times the cores. Over the whole threads-only line the cost per iteration spans a factor of 1.26. Over the ranks-only line it spans 95.4. The OpenMP backend parallelises only part of the execution path, and the serial remainder bounds the achievable speedup. Chapter 5 returns to this with profiler evidence.

The MPI-only line from the same baseline reaches 8.1s on 64 ranks, a factor of 30. Its rank counts differ, so by Section 3.3 the wall-clock figure is indicative rather than a measurement of parallel efficiency. The 64-rank run performs *more* iterations than the baseline (7181 against 6700), so iteration count cannot account for any of the gap. Distributing this problem across the node works, and threading it does not.

Cost per iteration also falls monotonically as ranks replace threads at a fixed core count, as Table 3.2 shows: the trend is in the rank count, with no threshold at any particular thread count.

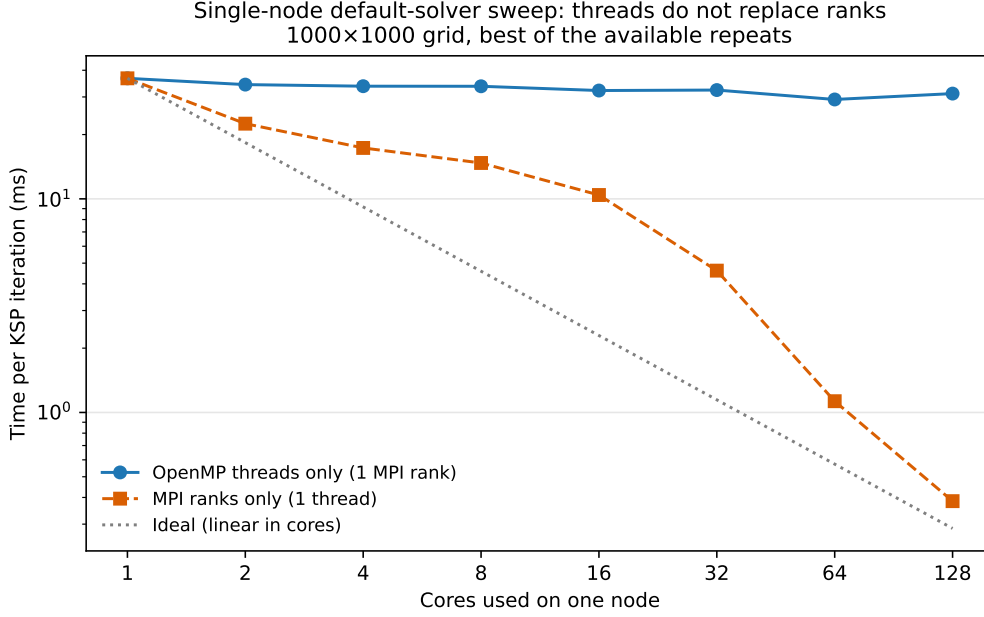


Figure 3.2: Single-node default-solver sweep on a 1000×1000 grid. Adding OpenMP threads to one MPI rank leaves the cost per iteration almost unchanged. Adding MPI ranks at one thread each reduces it by two orders of magnitude. Both lines start from the same one-rank, one-thread run.

Table 3.2: Cost per KSP iteration for the eight fully populated single-node layouts (ranks $\times$ threads = 128) on a 1000×1000 grid. The trend is monotone in the rank count, with no threshold at any particular thread count. The 128×1 figure is measured over its non-converged 10,000 iterations, which still gives a valid per-iteration cost.

Layout	Iterations	Repeats	ms per iteration	Converged
1×128	6700	1	31.10	yes
2×64	3715	1	14.42	yes
4×32	2825	1	7.24	yes
8×16	4085	1	3.67	yes
16×8	8179	2	1.89	yes
32×4	6460	2	0.99	yes
64×2	7181	2	0.61	yes
128×1	10000	2	0.39	no

3.5.2 The Same Bound on the Production Configuration

The sweep above cannot rank layouts with different rank counts, so the restriction was re-tested on the production configuration before it was applied to the formal campaigns. A separate multi-node campaign ran the same `arch-omp-libsci-rome-opt` build, the same `ex2_repeat` driver, and the same CG+GAMG options as Chapter 4, at 2, 4 and 8 nodes, and included the two layouts this rule excludes: 8×16 and 4×32 . Table 3.3 gives the result. In all nine size-node cells the time per iteration increases monotonically with threads per rank, and the excluded layouts are not marginal: 8×16 costs 1.8–3.8 times the best layout in its cell and 4×32 costs 3.0–7.4 times. The rule therefore separates factor-level penalties from the few-per-cent differences among the admitted layouts.

One run cannot resolve the 0.5% difference between 64×2 and 32×4 at 2m on eight nodes.

Table 3.3: KSP time per iteration for five full-node layouts in the production configuration. Each cell has one run and every row rises with threads per rank. The campaign has no 128×1 cell and bounds the search space rather than ranking the admitted layouts.

Scale	Nodes	Unknowns/core	ms per KSP iteration					Iterations
			64×2	32×4	16×8	8×16	4×32	
2m	2	7,810	1.75	2.25	3.52	6.08	11.82	9
2m	4	3,905	1.41	1.49	2.27	3.50	6.34	8–9
2m	8	1,953	1.30	1.30	1.64	2.32	3.88	8–9
4m	2	15,625	3.17	4.06	6.79	12.06	23.60	9–10
4m	4	7,813	2.13	2.52	3.80	6.46	12.39	8–9
4m	8	3,906	1.78	1.90	2.45	3.79	7.56	8–9
10m	2	39,006	11.79	13.25	20.01	30.68	61.51	9–10
10m	4	19,503	4.56	5.61	9.00	15.59	30.70	9–10
10m	8	9,752	3.10	3.28	5.05	8.65	17.03	9–10

The formal campaigns rank the admitted layouts. This campaign only excludes 16 and 32 threads per rank. The study therefore retains 1, 2, 4 and 8.

3.6 Job Configuration and Thread Placement

Each rank and its threads are pinned to a contiguous set of cores with:

```
OMP_PLACES=cores    OMP_PROC_BIND=close
srun --hint=nomultithread --distribution=block:block --exact
```

Each setting is part of the layout definition. `OMP_PLACES=cores` makes a place one physical core and prevents migration. `OMP_PROC_BIND=close` places a rank’s threads on adjacent cores. `--hint=nomultithread` selects 128 physical cores rather than 256 SMT siblings. `--distribution=block:block` assigns consecutive ranks to consecutive cores, while `--exact` restricts each task to its requested `--cpus-per-task` cores. Under the measured topology of Section 2.4, a 32×4 rank occupies one four-core shared-L3 domain and a 16×8 rank occupies two. No tested layout places a rank across two 16-core NUMA regions. Fixed pinning prevents placement variation from obscuring the few-per-cent layout effects under study.

3.7 Metrics

The primary timing is the maximum-rank `KSPSolve` time in the `RepeatedSolves` stage. If this event has count $c = 19$, time t , and the grid has N unknowns, steady-state throughput is

$$Q = \frac{Nc}{t}.$$

The analysis also reports $t/(cI)$, where I is the final KSP iteration count. Formal repeats are aggregated by their median. Whiskers show the observed minimum and maximum from three runs. Scaling is computed per layout against that layout’s smallest admitted node count. For a layout ℓ at n nodes,

$$S_\ell(n) = \frac{T_\ell(n_0)}{T_\ell(n)}, \quad E_\ell(n) = \frac{S_\ell(n)}{n/n_0},$$

where n_0 is the smallest node count run for that problem size.

Two percentage conventions are kept distinct. For hybrid throughput Q_h and flat-MPI throughput Q_m , the throughput gain is $Q_h/Q_m - 1$. For their times t_h and t_m , the time saving is $1 - t_h/t_m$. They describe the same pair of runs but are not numerically equal. Every percentage in the results names the convention it uses. Time per iteration is used to separate convergence-count differences, not to claim identical multigrid work. The three-run minimum and maximum describe only the variation observed in this campaign. They are neither a confidence interval nor a significance test. When two observed ranges overlap, the text treats the result as unresolved instead of selecting the smaller median. When they do not overlap, the result supports a difference within this sample but still does not establish a universal ordering beyond the tested configuration. Every direct layout comparison also holds problem size, node count, total physical cores, solver options and convergence tolerance fixed. Comparisons across scaling regimes remain separate even when they share a size–node pair.

3.8 Superseded Campaigns

Two earlier campaigns were discarded before the formal study. Their defects are retained here because neither was visible in the resulting plots, and each motivated a check now enforced by the analysis pipeline.

The first round did not use its requested tolerance. All four drivers inherit a mesh-dependent relative tolerance from upstream `ex2` before `KSPSetFromOptions` is called: $\text{rtol} = 10^{-2}/((m+1)(n+1))$ in 2D and the corresponding product in 3D (`src/ex2_repeat.c:109` and siblings). The exploratory job script overrode it with a double-dashed option, `--ksp_rtol 1e-5`, while neighbouring single-dashed options took effect. Iterations rose by 39–62% between the 800² and 1000² campaigns. The compute-node test in Table 3.4 shows why.

Table 3.4: Relative tolerance actually used by `ex2` on a 100×100 grid, read from `-ksp_view`, for the three option styles. The double dash is silently ignored and the mesh-dependent default survives. `-options_left` names the unconsumed option explicitly.

Run	Option as passed	relative=	Iterations
G13-a	<code>--ksp_rtol 1e-5</code>	9.80×10^{-7}	84
G13-b	<code>-ksp_rtol 1e-5</code>	1×10^{-5}	65
G13-c	none	9.80×10^{-7}	84

The double-dashed option was never consumed: run G13-a is identical to the no-override control in both tolerance and iteration count, and PETSc’s `-options_left` report names `--ksp_rtol` as unused. The effective tolerance of those campaigns therefore tightened as $1/N$ with the mesh, reaching 2.4×10^{-10} at 6400². That is the second half of the non-convergence reported in Section 3.3: the runs were asked for a solution five orders of magnitude more accurate than intended, with a preconditioner that also weakened as the rank count grew. The exploratory configuration differs from the production one in *both* solver and tolerance, which is why no default-versus-GAMG performance comparison is drawn anywhere in this dissertation. Section 6.2 restates the limit.

The second round used CG+GAMG on multiple nodes, but the requested layouts did not run. Every PETSc log states the process count actually initialised in its header. That count disagrees with the filename in 141 of 142 2D logs and all 161 3D logs, usually reporting one process where 16, 32, 64 or 128 were requested. The filename records intent, not execution, so two differently labelled layouts may both have run with one process. Their 20–50-fold slower solve times are consistent with that diagnosis.

Any layout comparison from these logs is therefore void, including the earlier claim that 32×4 was fastest at every size. The campaign also timed one setup-dominated solve: `PCSetUp`

accounts for a median 69% of total time and is $3.3\times$ the `KSPSolve` time. Seventy-two of 88 configurations have one run, so no observed range exists.

Both failures produced plausible output while filenames described intent rather than execution. The validation contract in Section 3.9 therefore checks the reported build, options, process count and thread count, and fails on a mismatch instead of silently dropping a row. Superseded logs remain as provenance (Appendix A). The exploratory single-node sweep is unaffected by the launcher defect and is used only at fixed rank count or per iteration, where its tolerance and preconditioner are common to the comparison.

3.9 Reproducibility

Every result in Chapter 4 and Chapter 5 is generated from raw job output by a campaign script. Appendix A lists the commands. Three conventions make the chain auditable. First, raw `-log_view` output is retained unmodified, while derived CSVs, tables and figures are regenerated rather than hand-edited. Second, parsers trust log contents rather than filenames. They check the reported build, the process and thread counts PETSc saw, solve and event counts, and the linked library. A mismatch raises instead of being dropped. Third, scripts assert properties used in the prose, so a data change that invalidates a claim also fails the pipeline.

The generator for Table 3.1, for example, refuses to write the table unless iteration count is constant across thread counts at every fixed rank count. Thirty unit tests cover parsing and campaign completeness. The limits remain explicit: three repeats support an observed range but not a significance claim, and the exposure shares in Section 5.4 are indicators rather than an additive time budget.

The MAP analysis follows the same contract. It reads each compressed `.map` XML file directly, validates the recorded `srun` layout, rejects profiles whose frames were not resolved to libraries, and asserts the ordering used in the text. This avoids a licensed GUI export step and preserves the profile file as the source evidence.

Chapter 4

2D and 3D Problems

4.1 Experimental Setup

Each dimension has two full-node campaigns using the platform, build, drivers, solver, placement and measurement contract of Chapter 3. Every node is populated with all 128 physical cores. The campaigns differ only in the scaling regime: weak scaling grows the problem with node count to hold local work approximately constant, whereas strong scaling holds one problem fixed to reveal saturation. Table 4.1 summarises both designs.

Table 4.1: Grids and node counts. Weak-scaling points pass the unknowns-per-core filter of Section 3.4. The fixed-size campaign runs the 20m problem over all nodes.

Scale	2D grid	2D unknowns	3D grid	3D unknowns	Weak-scaling nodes	Fixed-size nodes
5m	2240 ²	5,017,600	171 ³	5,000,211	1	—
10m	3160 ²	9,985,600	215 ³	9,938,375	1, 2	—
20m	4500 ²	20,250,000	273 ³	20,346,417	2, 4	1, 2, 4, 8, 16, 32
41m	6400 ²	40,960,000	345 ³	41,063,625	4, 8	—
82m	9073 ²	82,319,329	435 ³	82,312,875	8, 16	—
165m	12828 ²	164,557,584	548 ³	164,566,592	16, 32	—

The weak campaign contains 11 size–node pairs and the fixed campaign six. Every pair uses four layouts and three repeats. Before entering a median, each log is checked for its reported build, actual process and thread counts, 20 converged solves, 19 timed **RepeatedSolves** events, preconditioner reuse and linked LibSci path. All 280 weak-scaling logs and 144 accepted fixed-size logs pass. The 140 retrieved 3D source-CSV rows also match independently parsed logs one for one. Eight earlier 20m pilot logs per dimension remain as provenance but are excluded from all medians and figures.

The largest three-repeat min–max spreads are 4.71% in 2D and 3.83% in 3D for weak scaling, and 10.77% and 11.09% for the shorter fixed-size solves. Smaller layout gaps are reported as ties.

4.1.1 Weak-Scaling Chains

The filter $25,000 \leq N/C \leq 100,000$ couples problem size to node count and leaves only one or two node counts per scale. A per-size panel would therefore contain at most two points. The same 11 admitted pairs instead form two near-constant-work chains (Table 4.2), permitting a five- or six-point view of concurrency cost without changing the underlying measurements.

The six- and five-point chains remain within 15% of their nominal bands. They are reported as weak scaling, while adjacent node counts at a fixed size remain local strong-scaling observations. Ideal weak scaling holds time per iteration constant. Growth along a row measures concurrency

Table 4.2: The 11 admitted pairs arranged as two weak-scaling chains with near-constant unknowns per core.

Band	Size at node count					
	1	2	4	8	16	32
$\sim 40\text{k}$ unknowns/core	5m	10m	20m	41m	82m	165m
$\sim 80\text{k}$ unknowns/core	10m	20m	41m	82m	165m	—

cost at approximately fixed local work. The two readings share data but answer different questions and are not pooled into a new statistic.

4.1.2 Strong-Scaling Design

The adjacent-node measurements give only one strong-scaling step per size and cannot show where a curve turns. A matched extension therefore fixes the 20m problems and runs nodes $\{1, 2, 4, 8, 16, 32\}$ for every layout, without changing the solver or repeated-solve contract. At 32 nodes only about 4.9 thousand unknowns remain per core, well outside the 25–100 thousand admitted by Section 3.4. This deliberately over-decomposed final point exists to show where adding nodes stops helping, and should not be read as an efficient operating range.

The 2- and 4-node anchors repeat size–node pairs collected in the earlier weak campaign and test for a shift between measurement sessions. The 2D medians differ by at most 2.2%, with six of eight ranges overlapping. The 3D reruns are 0.3–3.1% slower and none of the eight ranges overlaps, so the two sessions are reported separately rather than pooled.

The 3D 1-, 2- and 32-node points were also submitted twice. The analysis selects the newer complete batch of three repeats and retains 36 earlier logs as superseded provenance. The batches agree within 0.78% at one and two nodes and differ by -4.7% to $+5.1\%$ at 32, where solve times are only tens of milliseconds. Both select 16×8 at 32 nodes, ahead of flat MPI by $3.81\times$ and $4.00\times$. The saturation conclusion therefore does not depend on which complete batch is used, but the batches are not combined into a six-repeat median.

4.2 Weak Scaling at Constant Work per Core

Both dimensions use the same 11 pairs, four layouts and three repeats.

4.2.1 2D

Throughput rises almost linearly along both chains, while layouts differ by only a few per cent on that scale (Figure B.1). Figure 4.1 instead divides time per KSP iteration by flat MPI at the same pair, removing scale and the 9–13 GAMG iterations.

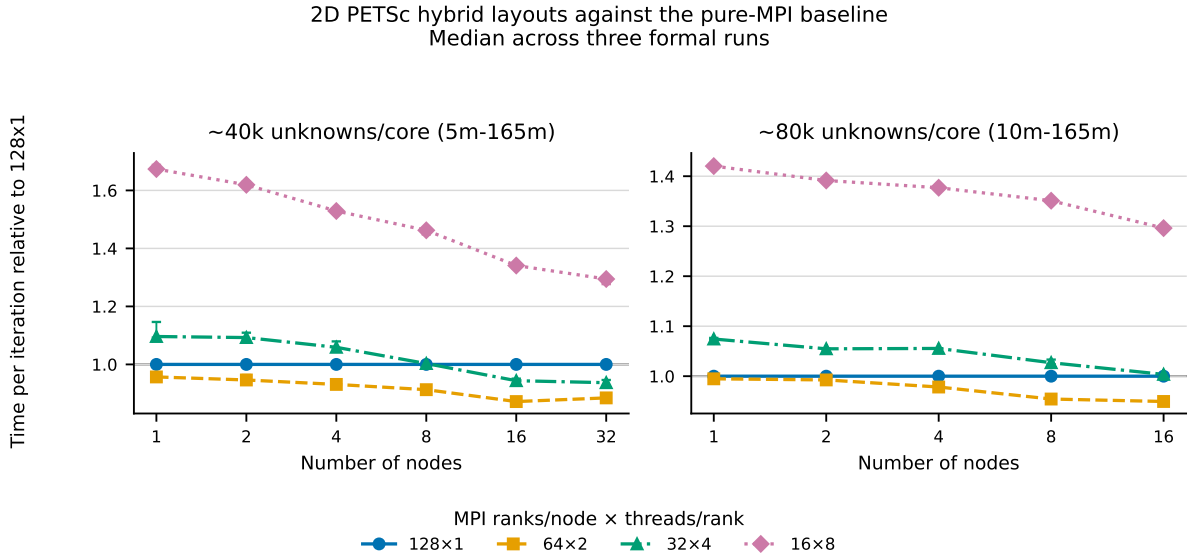


Figure 4.1: 2D time per KSP iteration relative to the 128×1 baseline at the same size–node pair. Values below 1 mean the hybrid layout is faster. Whiskers propagate the observed range of three formal runs.

Four of the six ratio series are monotone, meaning that the hybrid layout gains on flat MPI at every step up in node count. Along the $\sim 40k$ chain, the 64×2 ratio falls from 0.957 at one

node to 0.872 at 16 before a small reversal to 0.885 at 32, and 32×4 falls from 1.096 to 0.937. Along the $\sim 80k$ chain all three hybrid ratios decrease at every step. Thus 64×2 is faster per iteration at all 11 pairs, whereas 32×4 crosses the baseline: it costs 9.6% more at one node, reaches parity at eight (1.003), and costs 6.3% less at 32. A small-node ranking therefore does not transfer unchanged.

Raw throughput also includes convergence differences. 64×2 leads at eight of 11 pairs. Two of the three flat-MPI leads coincide with one fewer GAMG iteration, and the third is a 0.4% tie with overlapping ranges. At 165m on 32 nodes it reaches 794.8 million equations/s, 13.1% above flat MPI. Its maxima are a 14.7% throughput gain and a 12.8% time saving at 82m on 16 nodes.

16×8 has the lowest throughput at all 11 pairs, although its deficit narrows from 46.2% to 22.7%.

Figure 4.2 shows the corresponding absolute time per iteration.

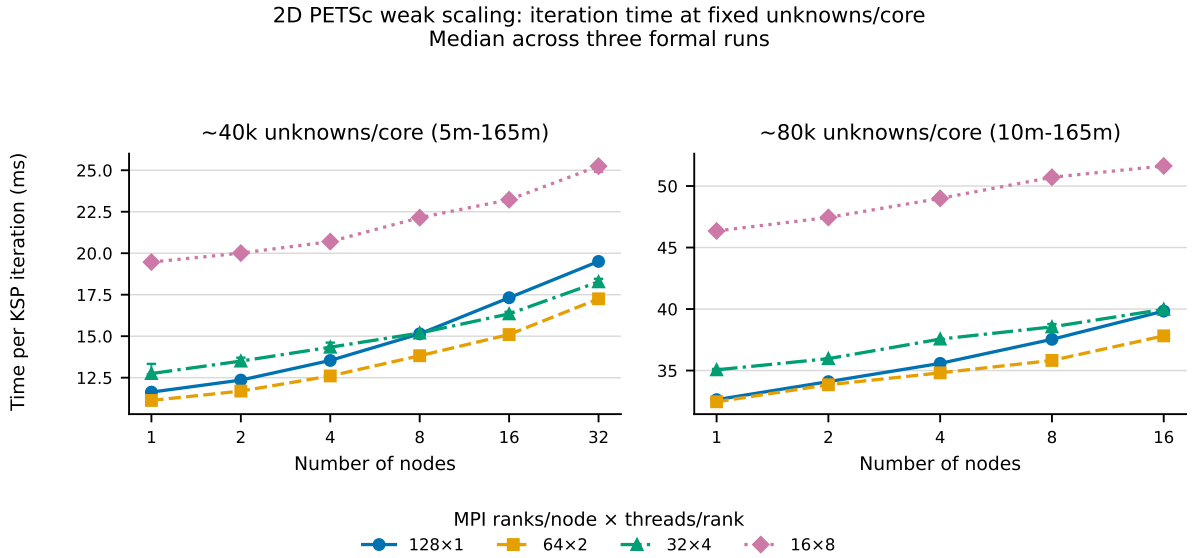


Figure 4.2: Median 2D time per KSP iteration along the two unknowns/core chains. A horizontal line would be ideal weak scaling.

4.2.2 3D

The 3D campaign uses the same design and takes six or seven KSP iterations. As in 2D, throughput is near-linear, and the layouts are easier to tell apart once time is normalised per iteration (Figures 4.3 and B.2).

The 3D advantage is larger but less ordered. At 41m on four nodes, 32×4 gives the largest time saving (24.9%) and throughput gain (33.2%); the corresponding 2D maxima are 12.8% and 14.7%. These percentages differ because one compares time and the other rate. Twenty-two of 33 hybrid measurements beat flat MPI per iteration, against 13 of 33 in 2D.

All six 3D ratio series are non-monotone. On the $\sim 80k$ chain hybrid layouts are strongest at two and four nodes, with 32×4 reaching 0.784 and 0.751, then reach or exceed parity at 16, where flat MPI is fastest per iteration. On the $\sim 40k$ chain, 64×2 recovers from 1.022 at eight nodes to 0.768 at 32. The advantage is therefore larger than in 2D but less transferable between node counts.

Raw throughput selects 32×4 at six pairs, 64×2 at four, and flat MPI at one. At 165m on 32 nodes, 64×2 reaches 510.4 million equations/s, 30.1% above flat MPI. 16×8 wins no pair but beats flat MPI at five.

Figure 4.4 gives absolute time per iteration.

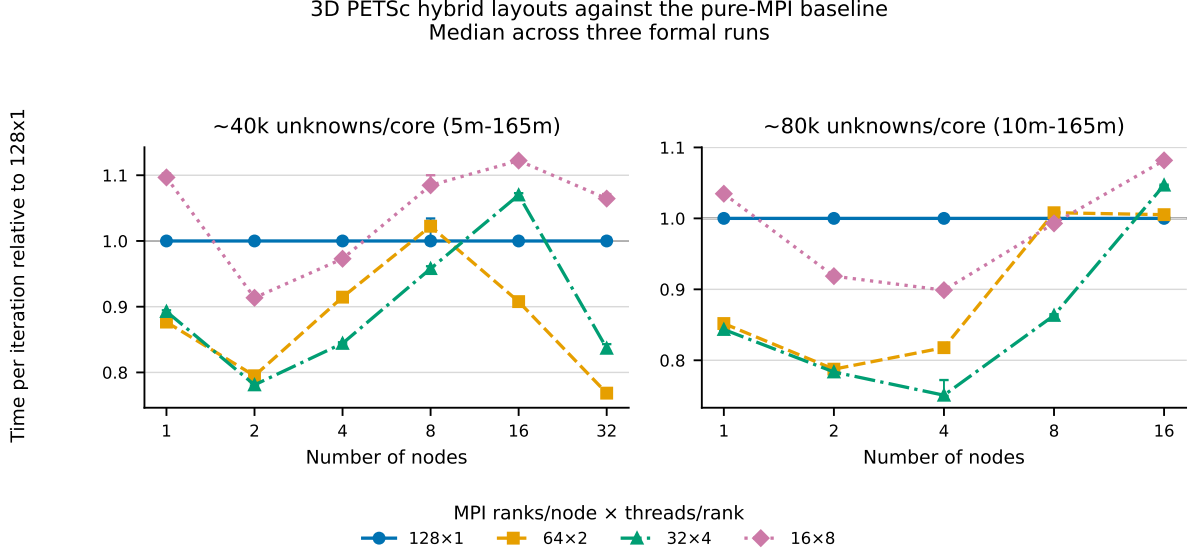


Figure 4.3: 3D time per KSP iteration relative to the 128×1 baseline at the same size-node pair. Unlike the 2D case in Figure 4.1, no series is monotone.

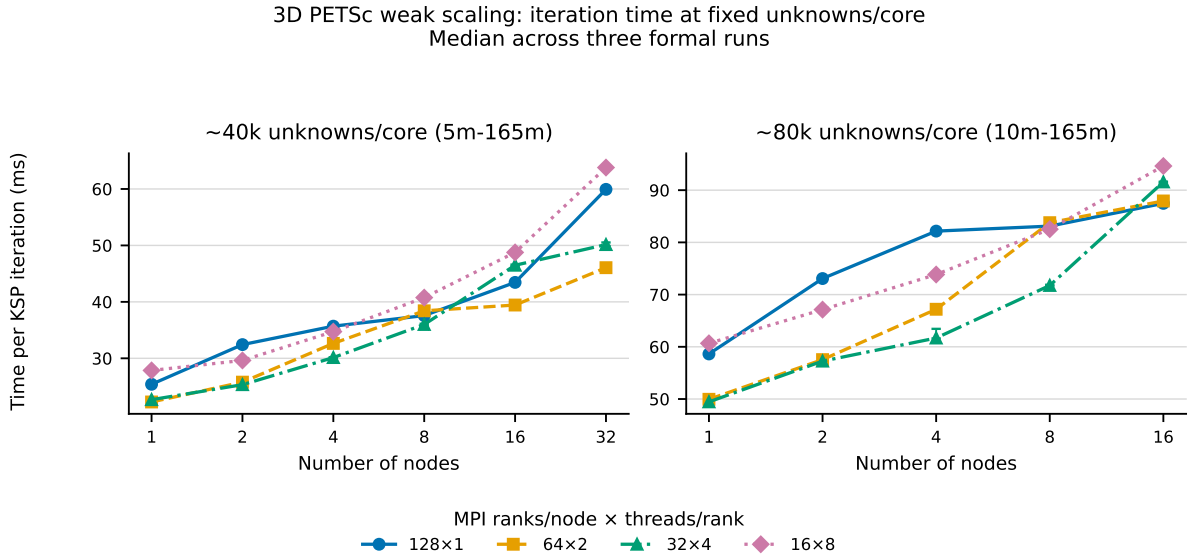


Figure 4.4: Median 3D time per KSP iteration along the two unknowns/core chains.

4.2.3 Layout Ranking and Efficiency

Table 4.3 gives the fastest layout and its margin over flat MPI at every pair.

The 2D winner changes twice across the 11 rows. The 3D winner changes five times. This matches the ordered 2D and scale-dependent 3D ratio trends.

Taking each layout's own single-node time as its baseline gives the weak-scaling efficiencies in Table 4.4.

In 2D, efficiency rises with threads per rank in both chains. This tracks the falling ratio curves but does not rank absolute speed: 16×8 has the best efficiency and the worst time at every point. A metric based on each layout's own baseline can reward a poor baseline.

The 3D efficiencies have no thread-count ordering: 64×2 leads the $\sim 40k$ chain, whereas flat MPI leads the $\sim 80k$ chain. The 2D pattern does not transfer.

Table 4.3: Fastest layout by median throughput over three repeats, its margin over flat MPI, and time per iteration. The 2D 41m/8-node row is a tie because ranges overlap.

Scale	Nodes	Cores	Best layout	M eq/s	vs. MPI	ms/iteration
<i>2D, five-point stencil</i>						
5m	1	128	128×1	47.9	0.0%	11.63
10m	1	128	128×1	34.0	0.0%	32.63
10m	2	256	64×2	94.9	5.7%	11.69
20m	2	256	64×2	59.8	0.8%	33.84
20m	4	512	64×2	160.7	7.4%	12.60
41m	4	512	64×2	107.0	2.2%	34.81
41m	8	1024	128×1	270.5	0.0%	15.14
82m	8	1024	64×2	208.9	4.8%	35.82
82m	16	2048	64×2	495.6	14.7%	15.10
165m	16	2048	64×2	362.6	5.3%	37.82
165m	32	4096	64×2	794.8	13.1%	17.25
<i>3D, seven-point stencil</i>						
5m	1	128	64×2	37.4	14.1%	22.26
10m	1	128	32×4	28.7	1.6%	49.45
10m	2	256	64×2	64.3	25.8%	25.78
20m	2	256	32×4	50.8	27.6%	57.26
20m	4	512	32×4	96.5	1.5%	30.13
41m	4	512	32×4	95.1	33.2%	61.68
41m	8	1024	32×4	163.0	4.4%	35.99
82m	8	1024	32×4	163.9	15.8%	71.75
82m	16	2048	64×2	298.2	10.2%	39.44
165m	16	2048	128×1	268.8	0.0%	87.47
165m	32	4096	64×2	510.4	30.1%	46.06

Table 4.4: Weak-scaling efficiency (%) at the largest node count of each chain, relative to the same layout’s one-node time. Each layout is measured against itself, so this quantifies degradation, not absolute speed.

Case	Chain and node count	Layout			
		128×1	64×2	32×4	16×8
2D	~40k, 32 nodes	59.6	64.5	69.8	77.1
2D	~80k, 16 nodes	81.9	85.8	87.7	89.8
3D	~40k, 32 nodes	42.4	48.3	45.2	43.7
3D	~80k, 16 nodes	67.0	56.8	54.0	64.1

4.3 Strong Scaling at Fixed Problem Size

The fixed-size campaign asks where additional nodes stop reducing runtime. Both dimensions use the same six node counts and four layouts.

4.3.1 2D

The five local 2D doublings give speedups of 2.00–3.08. Superlinear steps repeat within the three-run sample, but their cause is not identified.

All 2D fixed-size curves still improve at 32 nodes (Figure 4.5). The fastest-to-slowest spread grows from $1.47\times$ at one node to $1.81\times$ at 16, then narrows to $1.48\times$. 64×2 is fastest at nodes $\{2, 4, 8, 32\}$ and flat MPI at $\{1, 16\}$. Cumulative efficiency peaks at 191% for 64×2 at eight nodes; cache capacity and GAMG hierarchy changes remain untested explanations for this

superlinearity.

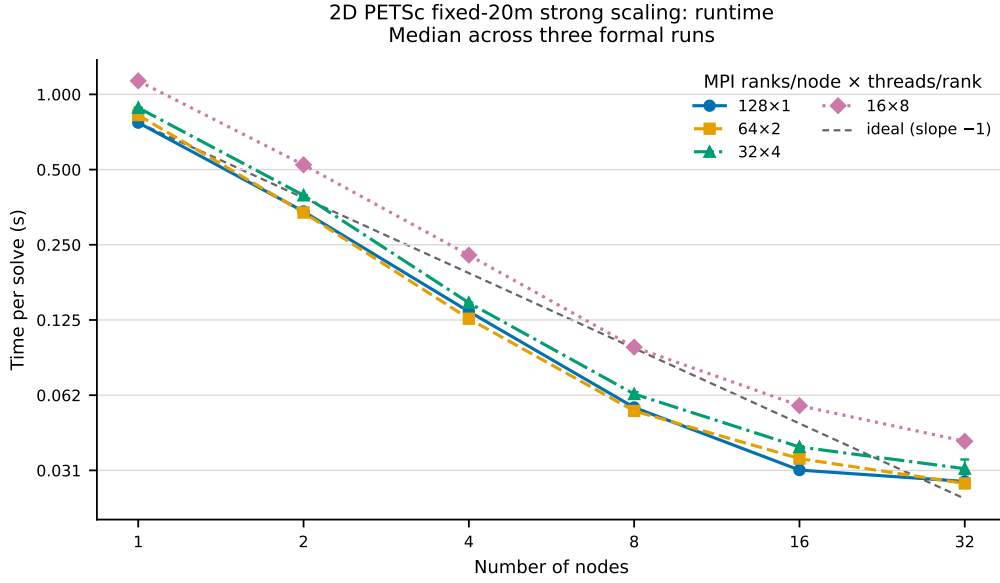


Figure 4.5: 2D time per solve at a fixed 20.25M unknowns, one to 32 nodes. Both axes are logarithmic, so ideal strong scaling is a straight line of slope -1 . Whiskers show the observed range of three formal runs.

4.3.2 3D

The five local 3D doublings give speedups of 1.46–2.39, without a superlinear step. At 165m, 64×2 gives $1.91\times$ from 16 to 32 nodes, against $1.46\times$ for MPI.

Three 3D layouts reach a minimum time and then get slower again within the measured range (Figure 4.6). Flat MPI reaches its minimum at eight nodes, 64×2 and 32×4 at 16. Only 16×8 still improves at 32 nodes, where its 0.0681s solve is $3.81\times$ faster than flat MPI on the same 4096 cores and 10% faster than 32×4 . The disjoint ranges support a layout difference at 32 nodes, but the campaign does not locate the node count at which 16×8 would itself stop improving.

Figure 4.7 measures each layout against its own one-node time, so it ranks scaling behaviour rather than speed. Section 4.3.3 reports both. The 2D curves do not turn, so their efficiencies remain in Table 4.6.

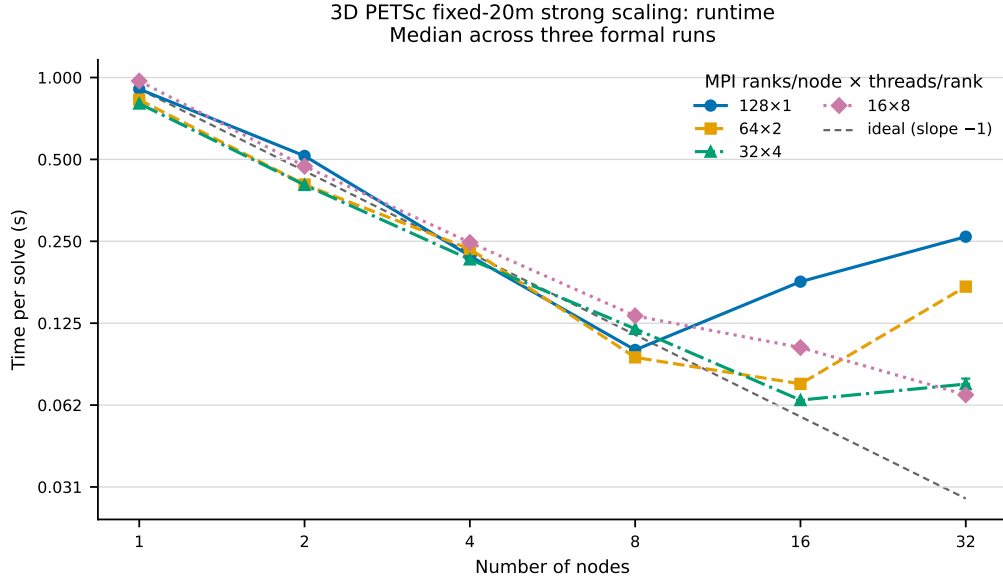


Figure 4.6: 3D time per solve at a fixed 20.35M unknowns. Flat MPI bottoms at eight nodes, 64×2 and 32×4 at 16, and only 16×8 still improves at 32.

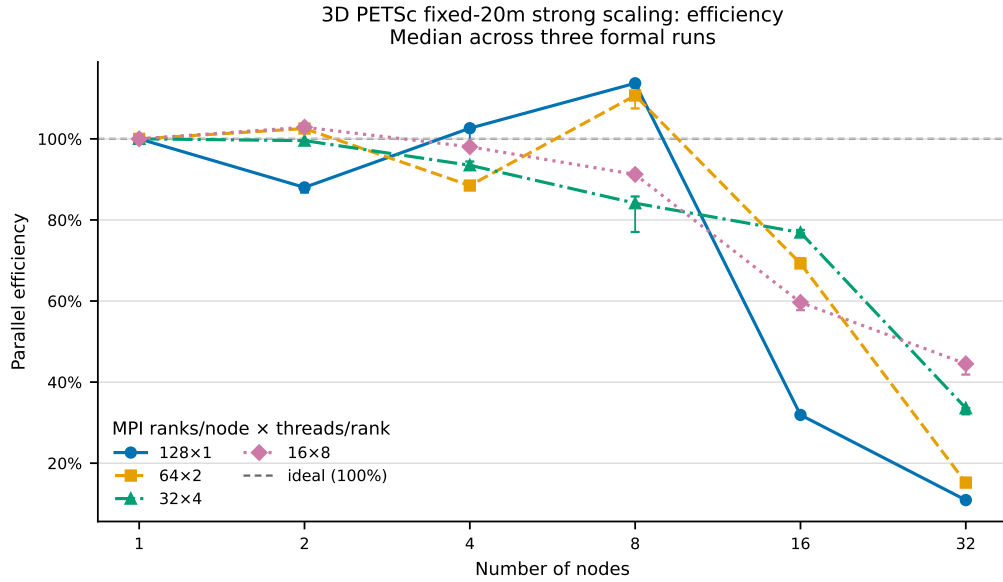


Figure 4.7: 3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. Because the baselines differ, this figure ranks scaling behaviour and not speed. Flat MPI falls to 10.9% at 32 nodes while 16×8 retains 44.5%.

4.3.3 Best Layout and the 32-Node Endpoint

Table 4.5 gives the fastest layout and its margin over flat MPI at each node count. The 2D margin stays below 6.9%; the 3D margin reaches 281.3% at 32 nodes. Table 4.6 then reports the far end of each curve on its own, the 32-node point at 4096 cores, which is where the layouts are furthest apart.

Table 4.5: Fastest layout for each fixed 20M problem, by three-run median time per solve, and its advantage over flat MPI.

Nodes	Cores	2D, 4500 ²			3D, 273 ³		
		Best	s/solve	vs. MPI	Best	s/solve	vs. MPI
1	128	128×1	0.7704	0.0%	32×4	0.8022	13.0%
2	256	64×2	0.3365	1.0%	32×4	0.4028	27.8%
4	512	64×2	0.1265	6.9%	32×4	0.2145	2.9%
8	1024	64×2	0.0541	3.3%	64×2	0.0936	6.5%
16	2048	128×1	0.0313	0.0%	32×4	0.0652	172.4%
32	4096	64×2	0.0277	1.9%	16×8	0.0681	281.3%

At 32 nodes, 2D speedups cluster at 27.3–29.8 because each layout uses its own one-node baseline, although the runtimes there differ by 45% (Table 4.6). In 3D the speedup and runtime rankings agree.

Table 4.6: The 32-node endpoint of the fixed-size curves. Speedup and cumulative efficiency are measured against the same layout’s one-node time, so they reward a poor baseline. The doubling efficiency is the 16-to-32-node step alone. Time per solve is the only column that compares layouts directly.

Case	Layout	s/solve	Speedup	Efficiency	16→32 step
2D	128×1	0.0282	27.30	85.3%	55.5%
2D	64×2	0.0277	29.83	93.2%	62.9%
2D	32×4	0.0317	27.85	87.0%	61.1%
2D	16×8	0.0409	27.71	86.6%	69.4%
3D	128×1	0.2599	3.49	10.9%	34.2%
3D	64×2	0.1703	4.87	15.2%	22.0%
3D	32×4	0.0747	10.75	33.6%	43.7%
3D	16×8	0.0681	14.24	44.5%	74.6%

4.4 Interpretation

Table 4.7 summarises the different trend shapes. The 3D gaps against flat MPI at 16 and 32 nodes exceed observed run-to-run spreads. The 1–7% 2D fixed-size margins do not, so individual 2D rankings remain indicative.

Four of six 2D ratio series are monotone and weak efficiency rises with thread count. No 3D series is monotone, and its preferred layout changes with scale and nodes. Scaling regime also matters: layout differences are percentage-level at constant work per core but reach $3.81\times$ for the fixed 3D problem at 32 nodes because layouts saturate at different measured points.

Because the stencils perform different work per equation, cross-dimensional comparisons use layout ordering, margins to flat MPI and trend shape, not absolute throughput.

Table 4.7: Cross-dimensional weak- and strong-scaling summary. Throughput gain and time saving use rate and time conventions respectively. A layout turns upward when its time after its measured minimum increases.

	2D	3D
<i>Constant work per core, 11 size-node pairs</i>		
Layout winning most points	64×2 (8 of 11)	32×4 (6 of 11)
Hybrid points faster per iteration	13 of 33	22 of 33
Largest throughput gain	14.7%	33.2%
Largest per-iteration saving	12.8%	24.9%
Ratio series monotone in nodes	4 of 6	0 of 6
Weak efficiency rises with threads	yes	no
<i>Fixed 20M problem, one to 32 nodes</i>		
Layouts whose time turns upward	none of 4	3 of 4
First to turn upward	—	128×1 , at 8 nodes
Speedup on the last doubling	1.11–1.39×	0.44–1.49×
Fastest at 32 nodes	64×2	16×8
Its lead over flat MPI	1.02×	3.81×
Fastest-to-slowest spread at 32 nodes	1.48×	3.81×

This chapter does not identify the mechanism behind either the improving 2D hybrid ratios or the poor absolute performance of 16×8 . Chapter 5 tests whether event exposure and OpenMP waiting track these observations. Chapter 6 converts the combined evidence into recommendations.

A layout recommendation must therefore name its scaling regime. 16×8 is last at constant work per core in both dimensions but first at 4096 cores for the fixed 3D problem. These measurements answer different questions.

Chapter 5

Profiling and Bottleneck Analysis

5.1 Why a Repeated-Solve Driver

In a single solve at 20.25M with 16×8 , `-log_view` gave `PCSetUp` 45% of time and `KSPSolve` 10%. `ex2_repeat` runs the solve inside a `RepeatedSolves` log stage, zeroes the initial guess before each solve, and reuses the preconditioner (`-ksp_reuse_preconditioner` and `-pc_gamg_reuse_interpolation true`). With 20 solves the stage held 69% of time and 93% of flops; within the stage `KSPSolve` was 100% of time; each of the 19 repeats took 11 iterations. The corrected scaling campaign uses this same warm-up and reuse design, so Chapter 4 compares steady-state solves rather than first-touch and hierarchy-construction costs.

5.2 Analysis Methodology

The method combines MAP 25.0.4 timelines with `-log_view %T` (share of stage time) and `%F` (share of stage flops). An event whose `%T` exceeds its `%F` spends the difference on communication, synchronisation, or memory traffic. Table 5.1 lists single-point data for the `RepeatedSolves` stage at 20.25M with 16×8 .

Table 5.1: Per-event time and flop shares, `RepeatedSolves` stage, 20.25M, 16×8 .

Event	%T	%F	GF/s
<code>MatMult</code>	39	55	30.5
<code>MatMultTranspose</code>	18	6	6.7
<code>VecTDot+VecNorm</code>	8	6	—
<code>PCApply</code> (agg.)	82	82	21.6

`MatMultTranspose` (GAMG restriction) has a `%T` three times its `%F` and a rate $4.6\times$ below `MatMult`. `VecTDot` and `VecNorm` are the CG global reductions.

The two instruments answer different questions. Uninstrumented `-log_view` runs cover flat MPI and every hybrid layout, so they provide layout comparisons and named PETSc event exposure. MAP samples call trees and assigns core-time to PETSc, MPI, OpenMP and libraries, but cannot launch the 128-rank baseline and perturbs runtime. It is therefore used to locate where hybrid cores are active or waiting, not to supply scaling times. Neither output is an additive decomposition of the other: PETSc events are nested and reported as per-rank maxima, whereas MAP shares partition sampled core-time. Agreement between their orderings is supporting evidence at the accounting level, not permission to sum the quantities or infer a hardware cause.

5.3 MAP Profiling Results

Two MAP campaigns profiled the same 20.25M 2D problem:

1. a baseline set on `arch-omp-opt` at 2, 8 and 16 nodes;
2. a matching set on the production `arch-omp-libsci-rome-opt` build at 2 and 8 nodes.

Neither contains a 128×1 cell, because MAP cannot launch 128 MPI ranks per node on this system, so flat MPI has `-log_view` evidence only. Of the twelve profiles, eleven are used. The 16-node 16×8 baseline profile is rejected on its own content: MAP resolved no library symbols in it, so it carries no cost information.

MAP works by photographing every core at a fixed interval and recording, for each one, the function it is executing and the library that function belongs to. The solve window keeps only those photographs in which at least one core is inside `KSPSolve` and none is still inside `PCSetUp`: the stretches of the run during which the program is solving and nothing is still building the GAMG hierarchy. Core-time is then the number of cores found in a given library, summed over the kept photographs, and the shares below are that count divided by the total over all libraries. The window is recovered from the profile itself, so it is a span of time and not the PETSc `RepeatedSolves` logging stage, and the two must not be quoted as the same quantity. Table 5.2 gives the resulting shares and Figure 5.1 plots them.

Table 5.2: Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. OpenMP is the GNU runtime’s spin and wait paths, that is threads with no work. MAP instrumentation inflates these runs, so the ordering between layouts is the result; the absolute values do not transfer to the uninstrumented campaigns.

Build	Nodes	Layout	PETSc	OpenMP	MPI	LibSci
LibSci	2	64×2	31.9%	36.5%	29.9%	1.0%
LibSci	2	32×4	51.6%	40.1%	5.8%	1.7%
LibSci	2	16×8	42.2%	53.5%	1.4%	2.1%
LibSci	8	64×2	4.5%	48.3%	46.7%	0.2%
LibSci	8	32×4	7.5%	69.3%	22.6%	0.3%
LibSci	8	16×8	15.6%	76.5%	6.2%	1.0%
Baseline	2	32×4	51.7%	40.8%	6.7%	0.0%
Baseline	2	16×8	40.2%	57.8%	1.4%	0.0%
Baseline	8	32×4	8.8%	68.6%	22.2%	0.0%
Baseline	8	16×8	16.0%	77.5%	6.0%	0.0%
Baseline	16	32×4	1.2%	69.8%	29.0%	0.0%

The two costs trade against each other. The MPI share falls as threads replace ranks: at eight nodes it goes 46.7%, 22.6% and 6.2% across 64×2 , 32×4 and 16×8 . That is the ordering Section 5.4 obtains from `-log_view` on the whole formal matrix, and MAP is consistent with it at the event-accounting level. Time in the OpenMP runtime runs the other way, 48.3%, 69.3% and 76.5%, and it is not useful work: the frames are `cpu_relax`, `do_spin` and `__futex_wait`, that is threads waiting for something to do. The two builds agree wherever they overlap. Across the four common cells the MPI shares differ by at most 0.9 percentage points and the OpenMP shares by at most 4.3, with no systematic sign.

MAP’s limits bound all of this. There is one profile per cell, so there is no observed range. The overhead is large and uneven, separately measured at +35% on the timed stage and roughly

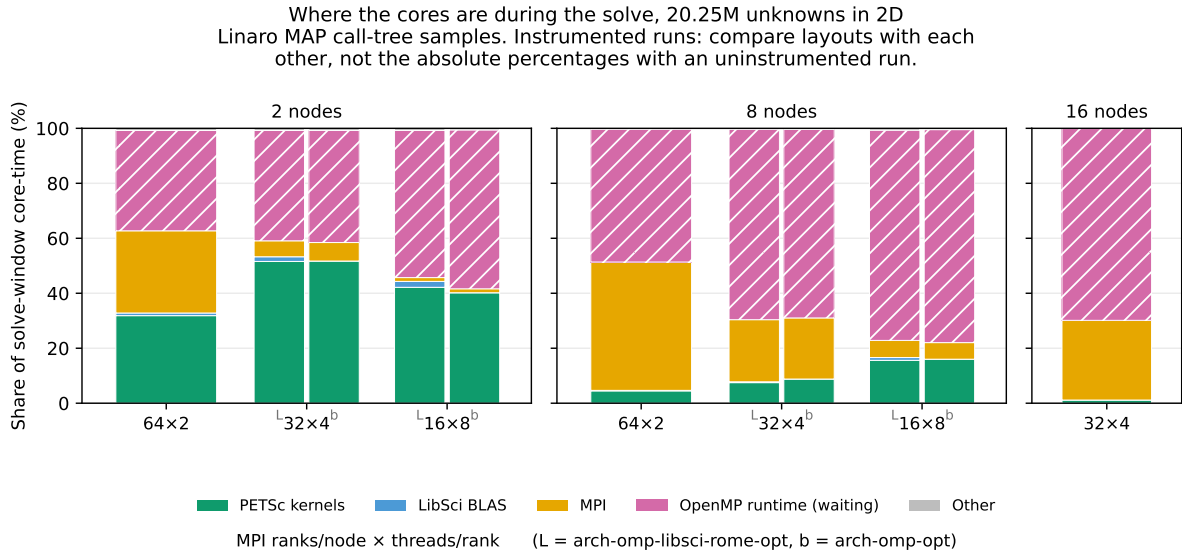


Figure 5.1: Where the cores are during the solve, from the MAP call trees. Within each panel the OpenMP waiting share grows and the MPI share shrinks as threads replace ranks, and both effects strengthen with node count. Paired bars are the two PETSc builds at the same configuration.

threefold on `VecScatterEnd`, and the OpenMP instrumentation perturbs the barrier behaviour being measured, so the absolute percentages overstate what an uninstrumented run would show.

Within those limits the table answers a question Chapter 4 could not, at the level of accounting rather than of mechanism. That chapter measured time per solve, in which the two sides of the trade are already netted against each other: it could show where the balance turns against more threads per rank, but not what those threads cost. Attributing each sample of core-time to the library owning its frame separates the two sides. The cost that grows with threads per rank, and that hybridisation has to repay before it pays off, is spent idle in the OpenMP runtime: what grows is time in `cpu_relax`, `do_spin` and `__futex_wait`, not time in the compute kernels. threads crosses a shared-L3 boundary but still fits inside one 16-core NUMA region, so no layout in this study puts a rank across two NUMA regions and the four cannot differ in that respect (Table 2.2).

Nor is the growth a placement effect, because it is present at a fixed layout, where a rank’s threads occupy the same cores against the same shared-L3 boundary at every node count. Holding 16×8 fixed and going from two nodes to eight raises the waiting share from 53.5% to 76.5% on the LibSci build and from 57.8% to 77.5% on the baseline, as the work per threaded region shrinks.

The other two features of the shape, monotonicity in the thread count and a maximum exactly where 16×8 performs worst, are consistent with that same explanation, that each threaded region holds less work. They are equally consistent with a cache explanation, in which a rank spanning two CCXs produces an imbalance that the same barriers collect. The profiles cannot separate the two, so why the threads wait is left to the hardware counters of Section 7.3, as Section 5.4.4 sets out. Section 5.4 shows the other half of the trade, on the full matrix and without a profiler.

The last observation is negative and worth stating. Threaded LibSci is linked into the production build, and PETSc’s sparse solve path barely calls it: BLAS accounts for at most 2.1% of solve-window core-time in any LibSci profile, and the baseline build, which is not linked against LibSci at all, reproduces the same OpenMP waiting growth without it. The threaded BLAS library therefore cannot account for the layout ordering measured in Chapter 4, which

confirms from profile data what Section 6.2 could previously only state as a limit of the evidence.

5.4 Bottleneck Identification

Section 5.3 leaves the hardware question open, but the `-log_view` event tables of the uninstrumented formal runs support a narrower question that Chapter 4 raised and could not answer: whether the layout effect tracks the share of the solve exposed to inter-process synchronisation. All 264 formal runs are parsed here, so the evidence covers the same matrix as the scaling results and carries no profiler overhead.

5.4.1 What is Measured

For each run, the time of the halo-exchange events (`VecScatterBegin`, `VecScatterEnd`) and of CG’s global-reduction events (`VecTDot`, `VecNorm`) is taken from the `RepeatedSolves` stage and divided by that stage’s `KSPSolve` time. Three limits apply and none is removable from `-log_view` alone. First, the synchronising events are logged *inside* `MatMult` and its relatives, so this is an exposure share and not a term in an additive time budget. Summing every event that is not itself a container for others gives 116–372% of `KSPSolve`, because `MatMult` and its relatives are containers for the halo exchange: the same wait is counted once in `MatMult`, once in the `VecScatter` pair, and once in `SFPack/SFUnpack`.

Second, each event time is a maximum over ranks, and different events may attain their maximum on different ranks, so the numerator sums maxima that need not be attained on the rank supplying the denominator. Third, `VecTDot` and `VecNorm` also perform local flops, so their time is not purely communication.

The quantity is nevertheless measured identically in every run, so it remains comparable across layouts and node counts.

5.4.2 Halo Exchange Dominates the Exposure, and Hybridisation Reduces It

Figure 5.2 and Figure 5.3 give the result. In both dimensions the share rises monotonically with node count for every layout and along each weak-scaling chain separately, and at 21 of the 22 size–node points it is strictly ordered by MPI rank count: fewer ranks, less exposure. The chains are panelled apart because they do not coincide: the smaller per-core problem is the more exposed one, by up to 15.5 percentage points in 2D and 17.2 in 3D at the same node count and layout, a difference comparable to the layout separation the figure exists to show. The plotted share is the sum of the two event classes, which sit on disjoint call paths: the halo exchange is reached from `MatMult` and its relatives, the reductions directly from CG.

The figures plot only that sum. Table 5.3 separates it at four representative points along the $\sim 40k$ unknowns/core chain in each dimension, the chain that reaches 32 nodes. Halo exchange is the larger term in all sixteen cells, by a factor of 1.7 to 2.9 in 2D and 2.9 to 5.7 in 3D. At 165m on 32 nodes flat MPI pays 34.3 percentage points to halo exchange in 2D and 70.9 in 3D, against 13.9 and 13.6 to the reductions. The totals these make up are the 48.2% and 84.6% at the right-hand end of Figure 5.2 and Figure 5.3. The 3D shares are far higher than the 2D shares at every point in the table, which is what the decomposition of Section 3.2 predicts: in 3D the $\pm mn$ coupling puts both k neighbours of every row off-rank once L drops below mn , while in 2D the matching $\pm m$ coupling never reaches more than $2m$ rows.

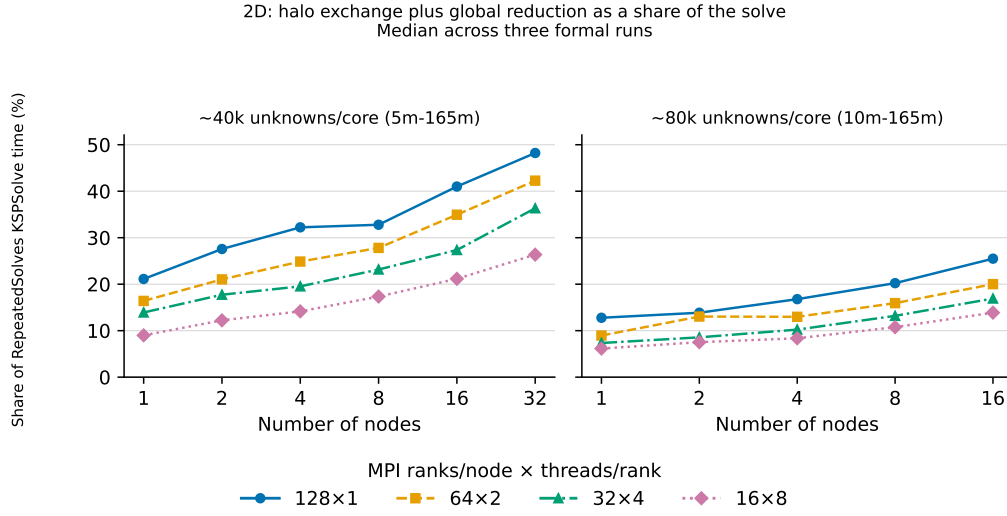


Figure 5.2: 2D share of `RepeatedSolves KSPSolve` time spent inside halo exchange and global reduction events, panelled by unknowns/core chain as in Chapter 4. Only the 40k chain reaches 32 nodes. Each point is the median of three repetitions. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.

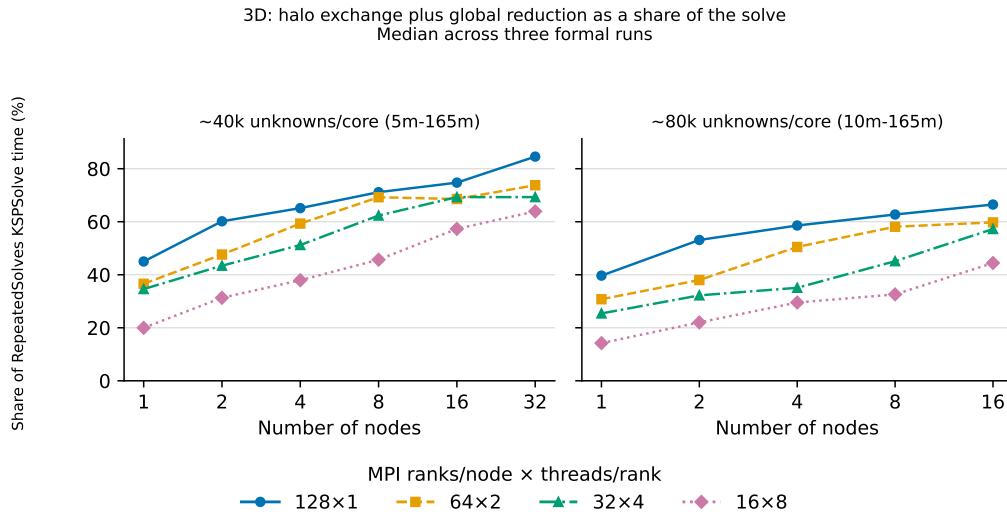


Figure 5.3: 3D share of `RepeatedSolves KSPSolve` time spent inside halo exchange and global reduction events, panelled and styled as in Figure 5.2. Note the vertical scale: flat MPI reaches 84.6% at the largest point.

Table 5.3: The exposure share of Figure 5.2 and Figure 5.3 split into its two event classes, at four representative points along the $\sim 40k$ unknowns/core chain of Table 4.2, for the 128×1 and 16×8 layouts. Halo is `VecScatterBegin` and `VecScatterEnd`; reduction is `VecTDot` and `VecNorm`. Their sum is the quantity plotted, so it is left to the figures. Each row is the repetition whose total is the median of its three, so both components come from the same run. The two are rounded independently and at one decimal need not sum to the plotted total (3D, 32 nodes, 128×1 : 70.9 and 13.6 against a plotted 84.6). Percentage points of `RepeatedSolves` `KSPSolve` time.

Case	Nodes (size)	128×1		16×8	
		Halo	Reduction	Halo	Reduction
2D	1 (5m)	15.7	5.4	6.1	2.9
2D	8 (41m)	23.4	9.4	11.2	6.1
2D	16 (82m)	28.6	12.4	13.6	7.6
2D	32 (165m)	34.3	13.9	16.4	9.9
3D	1 (5m)	35.1	9.9	15.3	4.6
3D	8 (41m)	53.0	18.1	38.8	6.8
3D	16 (82m)	59.0	15.8	46.5	10.8
3D	32 (165m)	70.9	13.6	52.4	11.5

5.4.3 Reduced Exposure is Necessary but not Sufficient

The ordering alone cannot explain the results, because 16×8 has the lowest exposure at every point of Figure 5.2 and Figure 5.3 and yet, in 2D, the worst time per iteration at all 11 of them. What does track performance is the *size* of the reduction, taken within a single layout, which gives the fits in Table 5.4.

Let S_ℓ be the plotted halo-plus-reduction share of layout ℓ and t_ℓ its time per KSP iteration, both at the same size-node point. The first two columns of Table 5.4 are the range, over the 11 points, of

$$\text{ExposureRemoved} = S_{128 \times 1} - S_\ell, \quad \text{TimeRatio} = \frac{t_\ell}{t_{128 \times 1}},$$

so $\text{ExposureRemoved} > 0$ means the layout is less exposed than flat MPI, and $\text{TimeRatio} < 1$ means it is faster. For example, ExposureRemoved is $(34.3 + 13.9) - (16.4 + 9.9) = 21.9$ points for 2D 16×8 at 165m on 32 nodes in Table 5.3.

Break-even, a measure defined here, is computed from a and b ,

$$\text{BreakEven} = \frac{1 - b}{a} = \frac{b - 1}{|a|}, \quad a < 0,$$

with none when $a \geq 0$. These two come from a least-squares fit over each layout's 11 pairs,

$$\text{TimeRatio} = a \text{ExposureRemoved} + b.$$

Here b is the intercept, the time ratio the line gives at $\text{ExposureRemoved} = 0$: what a layout would cost against flat MPI if its exposure share matched flat MPI's, so $b - 1$ is the cost of threading alone. The slope a is the change in TimeRatio per point of ExposureRemoved , so $|a|$ is how much of that cost one point pays back. fitted line drops below one exactly where ExposureRemoved passes BreakEven . The column is therefore a threshold: it is how much exposure a layout has to remove before threading pays for itself, and the lower it is, the sooner that happens. The fit is descriptive, so single points can fall on the wrong side of it.

The r column is the Pearson correlation, computed from the same 11 pairs,

$$r = \frac{\text{cov}(\text{ExposureRemoved}, \text{TimeRatio})}{\sigma_{\text{ExposureRemoved}} \sigma_{\text{TimeRatio}}},$$

their covariance over the product of their standard deviations. It shares the sign of a and says how closely they follow the line, so it measures how firm a break-even is rather than adding an effect of its own.

Table 5.4: ExposureRemoved against TimeRatio, both relative to flat MPI at the same size–node point, the first in percentage points. The break-even value is the exposure reduction the fitted line requires to offset threading overhead. a and b are that line’s slope and intercept, tabulated as 10^3a , so $(1 - b)/a$ reproduces the last column. *None* marks $a \geq 0$, where the line never descends to parity and no break-even exists. Eleven points per layout. The fits are descriptive only.

Layout	Exposure removed	Time ratio	r	10^3a	b	Break-even
<i>2D</i>						
64×2	0.8–7.4	0.872–0.995	−0.67	−15.29	1.0178	1.2
32×4	5.3–13.7	0.937–1.096	−0.59	−11.13	1.1305	11.7
16×8	6.3–21.9	1.294–1.674	+0.00	+0.08	1.4313	none
<i>3D</i>						
64×2	1.9–15.1	0.768–1.022	−0.86	−21.60	1.0618	2.9
32×4	5.5–23.5	0.751–1.070	−0.90	−17.47	1.1270	7.3
16×8	17.4–31.1	0.899–1.122	−0.83	−15.82	1.4226	26.7

Five of the six correlations are negative and three exceed 0.8 in magnitude: the more exposure a layout removes at a given point, the faster it is relative to flat MPI there. The a and b columns say why the threshold then climbs so steeply with threads per rank. Read down b , the cost of threading before any exposure is removed: it rises at every doubling of the thread count, and at 32×4 it is almost independent of the dimension, 1.1305 against 1.1270, although flat MPI’s exposure share differs between 2D and 3D by 36 points at the largest size–node point. That agreement is a property of the pooled fits and holds for this layout only. Section 5.4.4 sets out how far it survives when the two weak-scaling chains are separated. Read down $|a|$, the return on one point of ExposureRemoved: it falls at every step, and at all three layouts it is larger in 3D than in 2D. Break-even is their quotient, so a numerator that grows compounds with a denominator that shrinks and the thresholds rise faster than the threading cost alone. None of them is an extrapolation: every one, where it exists, falls inside the range of exposure the layout was actually observed to remove.

Figure 5.4 and Figure 5.5 plot the fits, 3D first because all three of its layouts behave the same way while the exception the argument turns on lies in 2D. Both split each layout’s 11 points into their two weak-scaling chains, filled against hollow, and draw the tabulated pooled fit solid across them. In 3D the three layouts occupy bands displaced from each other along the horizontal axis, each reaching a larger ExposureRemoved than the one before it and each needing more of it before crossing parity. The bands flatten as threads per rank rise, which is the a column, and within each band the solid line and its two chains stay together.

The relationship between ExposureRemoved and TimeRatio accounts descriptively for two patterns left by Chapter 4. For the first, holding a 3D layout and chain fixed while increasing the node count makes ExposureRemoved and TimeRatio reverse together at every step. Along the $\sim 40k$ chain in 3D, 32×4 has an ExposureRemoved of 10.4, 16.8, 13.9, 8.8, 5.5 and 15.3 points from one node to 32, and a TimeRatio of 0.893, 0.781, 0.844, 0.958, 1.070 and 0.837: five transitions, five reversals, including the recovery at 32 nodes where a 9.8-point rise in ExposureRemoved turns a 7.0% loss into a 16.3% gain. All 18 transitions of 64×2 and 32×4 across both 3D chains behave this way. The advantage therefore tracks the non-monotone ExposureRemoved within these measurements. This concerns the difference, not either share: along this chain 32×4 ’s own exposure rises monotonically from 34.6% to 69.3% and flat MPI’s from 45.0% to 84.6%, as Section 5.4.2 requires, and only the gap between them turns.

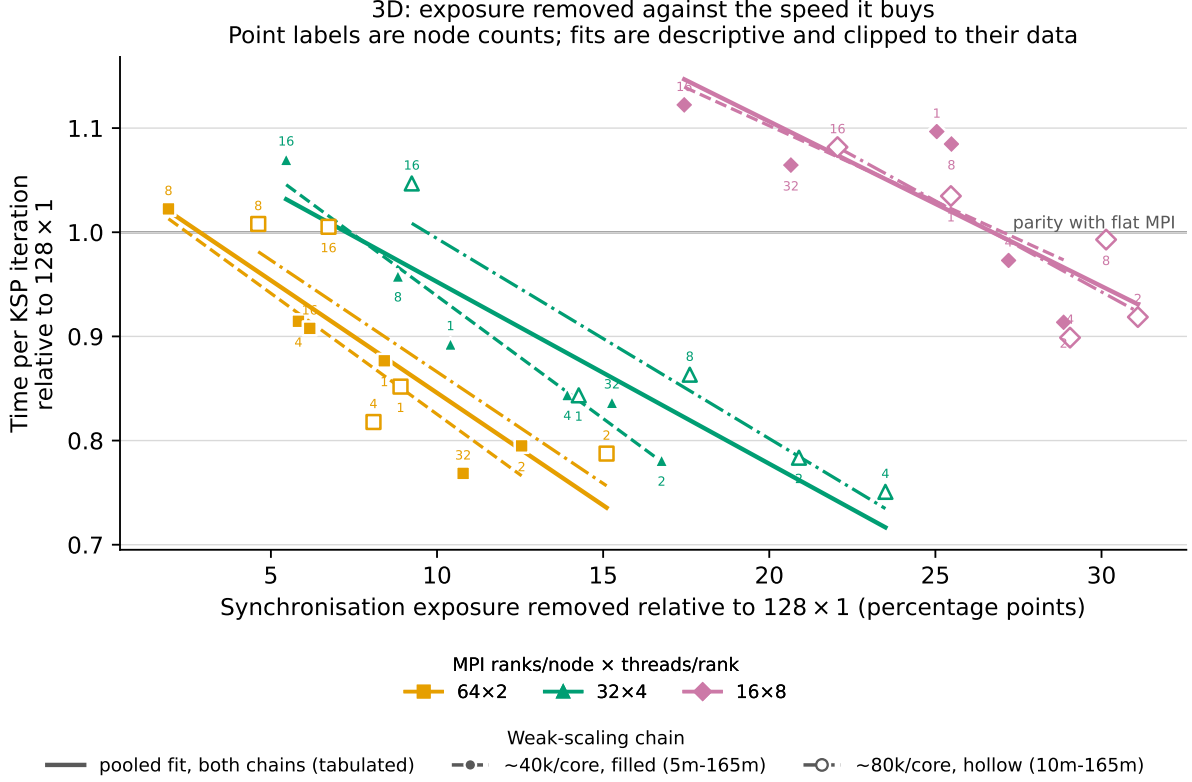


Figure 5.4: 3D exposure removed relative to flat MPI against the resulting time per iteration. Colour is the layout and point labels are node counts. The solid line is the pooled fit over all 11 points, the one Table 5.4 tabulates. The two broken lines split those points into their weak-scaling chains, dashed with filled markers for $\sim$ 40k unknowns/core and dash-dotted with hollow markers for $\sim$ 80k. In 3D the three stay together, every layout’s per-chain break-evens falling within 2.4 points of its pooled value. Each line is clipped to the range it was fitted to, so none is extrapolated. The fits are descriptive, not predictive.

The second is the threshold itself. In 2D, 32×4 brackets its own 11.7-point break-even along the $\sim$ 40k chain: an ExposureRemoved of 9.6 points at 41m on eight nodes for a TimeRatio of 1.003, then 13.7 points at 82m on 16 nodes for 0.944, crossing parity where the fit puts it. Along the $\sim$ 80k chain its ExposureRemoved never exceeds 8.6 points and it never crosses, bottoming at 1.004. The threshold holds in both directions.

The 2D 16×8 row is the counter-example, and Figure 5.5 shows what it consists of: its pooled a is positive, so the solid line lies flat and the table returns no break-even at all, while the two broken lines through the very same 11 points, one per chain, both descend steeply across it. The flatness is therefore an artefact of pooling. Taken separately the chains correlate at -0.91 over the six $\sim$ 40k points and -0.96 over the five $\sim$ 80k points, among the tightest fits in the study. What the layout lacks is therefore not a relationship between ExposureRemoved and speed but enough of the former: each chain’s line reaches parity only beyond the largest ExposureRemoved the layout ever attains, 21.9 points, so neither threshold can be stated without extrapolating and neither is ever reached. That it is slower than flat MPI by 29–67% at every one of its 11 points is a direct observation and needs no fit. Some other cost dominates.

Table 5.2 identifies that cost: at the same problem size and node counts, 16×8 spends more of its core-time waiting in the OpenMP runtime than any other layout, 76.5% against 48.3% for 64×2 at eight nodes. The two halves of the trade are therefore both measured, one on the full matrix without a profiler and the other on eleven profiled cells with one. What remains

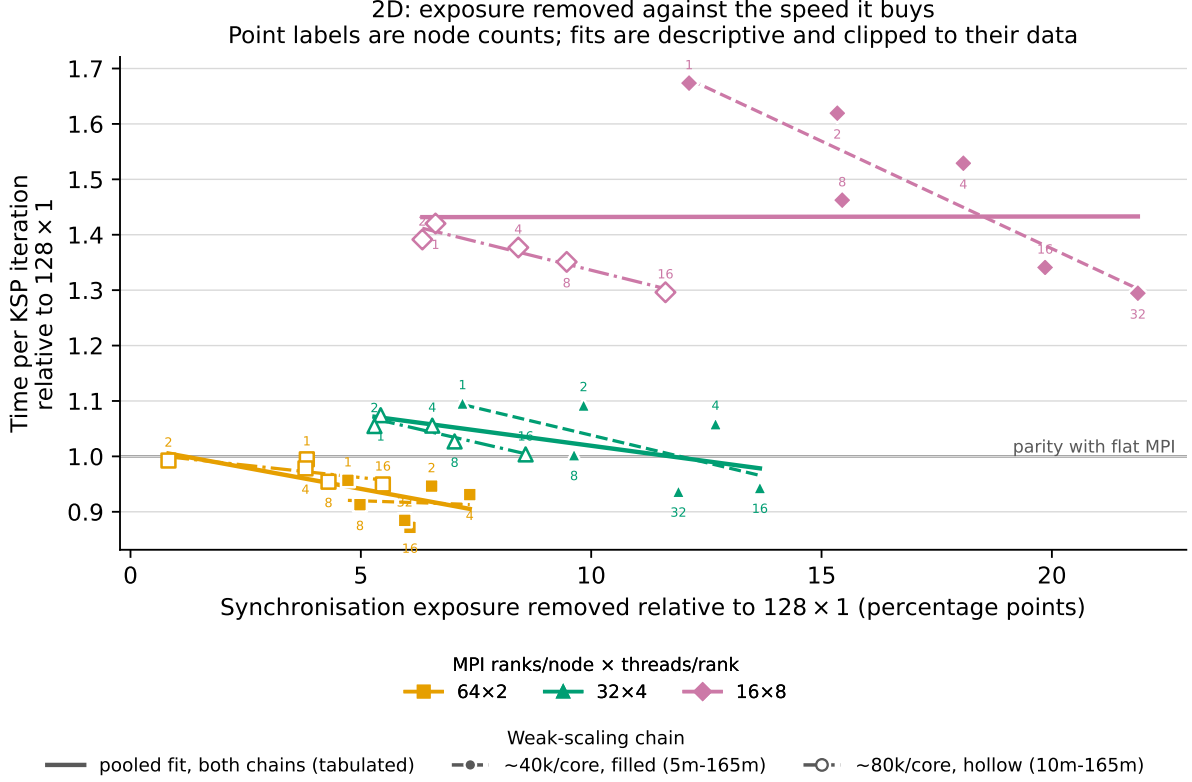


Figure 5.5: 2D exposure removed relative to flat MPI against the resulting time per iteration, styled as in Figure 5.4. Here the solid pooled fits are not summaries of their parts. For 16×8 the solid line lies flat while its two chains, correlating at -0.91 and -0.96 separately, descend steeply across it. 64×2 and 32×4 depart from their pooled fits as well. Note also that 16×8 lies above parity at every one of its 11 points, which is read off the markers and not off any fit.

unmeasured is their sum: the profiled and unprofiled quantities come from different instruments on different runs and cannot be added into a single budget.

5.4.4 What This Does and Does Not Establish

Pooling all 33 hybrid measurements of a dimension breaks the relationship between ExposureRemoved and TimeRatio, and breaks it differently in each dimension. In 2D the pooled correlation is $+0.58$, where the two layouts that have a break-even give -0.67 and -0.59 . In 3D it falls to $+0.10$ from three values that all exceed 0.8 in magnitude.

The first candidate is 16×8 , which has the largest ExposureRemoved and is also the slowest. That is enough in 3D: drop it and the pooled figure returns to -0.75 . It is not enough in 2D, where dropping it still leaves $+0.13$. There, 32×4 has a larger ExposureRemoved than 64×2 at all 11 points and is slower at all 11. The ordering between layouts runs against the ordering within them in both dimensions, and 16×8 is only the most extreme case of it. The relationship holds within a layout, so a pooled figure would mislead.

The two weak-scaling chains cannot be pooled either, and that is a second and independent effect: in 3D it does little harm, every layout's per-chain break-evens falling within 2.4 points of its pooled one, while in 2D none of the three layouts survives unchanged and 16×8 inverts the sign. Table 5.4 still reports pooled fits, because five and six points are too few to fit a chain on its own, and its columns should be read with that in mind. The b column agrees across dimensions only because the fits are pooled: split the chains and the agreement loosens at 32×4 and fails on the $\sim 40k$ chain at 16×8 . Neither pooling changes which layouts beat flat MPI, because that

is read off the measurements and not off a fit.

Within PETSc's event accounting, layout advantage tracks lower halo-exchange exposure, with global reduction secondary. MAP places the opposing sampled cost in OpenMP wait paths. Neither observation excludes a memory or cache mechanism underneath those software events. Section 6.1.4 names the candidates, and Section 7.3 gives the counter experiment.

Chapter 6

Discussion

6.1 Answering the Research Questions

The four questions of Section 1.2 are answered in turn. RQ1 and RQ2 rest on Chapter 4, RQ3 on Chapter 5, and RQ4 on the limits of both.

6.1.1 RQ1: Does the threaded backend reduce time to solution, and by how much?

Yes. Hybrid execution cuts steady-state solve time in both dimensions. What differs between 2D and 3D is how the gain behaves as the job grows, not whether there is one.

In 2D the gain grows with node count. Along both unknowns/core chains, every hybrid layout gains on flat MPI as the node count grows, and four of the six series gain at every step. 64×2 is faster per iteration than flat MPI at all 11 size-node points, and its margin widens from 4.3% at one node to 12.8% at 16. 32×4 starts 9.6% slower at one node and ends 6.3% faster at 32. A layout comparison made on one or two nodes therefore does not predict what happens on 16 or 32, which is the practical content of the 2D result.

In 3D the gain is larger but follows no such trend. The best hybrid gain in throughput is 33.2%, against 14.7% in 2D, and two-thirds of the hybrid points are faster per iteration than flat MPI. No series gains at every step. The advantage is largest at two to four nodes, is gone by eight to 16, and returns at the largest point. On raw throughput 32×4 wins six of the 11 points, 64×2 four, and flat MPI one. In 3D the best balance depends on both problem size and node count, which the 2D data do not show.

6.1.2 RQ2: At which balance, and how does the answer change with the regime?

At constant work per core the useful range is two to four threads per rank in both dimensions. Which of the two is better depends on the dimension: 64×2 everywhere in 2D, and either 32×4 or 64×2 in 3D depending on size and node count.

Holding the problem size fixed does not change which layouts are good at moderate scale. It changes what the choice costs, from a few per cent to whole factors, because the layouts differ in where they stop scaling. In 2D none of them stops. On 20.25M unknowns the four stay within a factor of 1.81 of each other from one node to 32, and all four are still getting faster at the end, the last doubling worth 1.11 to 1.39 $\times$. In 3D they come apart. On the matched 20.35M problem flat MPI stops improving after eight nodes, 64×2 and 32×4 after 16, and only 16×8 is still improving at 32. Flat MPI ends 2.6 $\times$ slower at 32 nodes than at its own best point, so at 4096 cores the gap to 16×8 reaches 3.81 $\times$.

Eight threads per rank does not run faster at a given scale. It keeps improving at a scale where the others have stopped. That is the whole case for it. 16×8 wins no raw-throughput

point in either weak-scaling campaign, and its good weak-scaling efficiency in 2D is not a point in its favour either, for the reason given in Section 6.2. Where it does earn its place is the fixed-size curves of Section 4.1.2. Drive the problem down to about 4.9 thousand unknowns per core, far below the 25–100 thousand the admission window of Section 3.4 allows, and 16×8 is the fastest 3D layout at 32 nodes and the only one of the four still improving. Flat MPI has by then been getting slower for two doublings, which is where the $3.81\times$ above comes from.

A layout recommendation means nothing without the scaling regime it was measured in. The same eleven size–node pairs, read at constant work per core, put the useful range at two to four threads and rank 16×8 last. The same solver and build on a fixed problem make 16×8 the only 3D layout still improving at 4096 cores.

Both throughput views are needed. Raw throughput is the time this solver configuration actually pays, and it includes any change in GAMG convergence. Time per iteration takes the iteration count out by holding the outer KSP count fixed, but even that does not make the GAMG hierarchy or the communication volume the same across decompositions.

6.1.3 RQ3: Which parts of the solve account for the difference?

Section 5.4 shows that hybrid layouts spend less time exposed to halo exchange and global reductions than flat MPI, at all 22 size–node points, and halo exchange is the larger of the two. Within one layout, how much exposure it removes tracks how much faster it is ($|r|$ up to 0.90). Layouts with more threads per rank have to remove more exposure before they break even. This is consistent with the useful two-to-four-thread range and the uneven 3D ordering, but does not establish their cause.

The other half of that trade is measured too, on eleven Linaro MAP profiles of the 20.25M 2D problem. Within the solve window the share of core-time spent waiting in the OpenMP runtime rises with threads per rank (48.3%, 69.3% and 76.5% at eight nodes) while the MPI share falls (46.7%, 22.6% and 6.2%). Both orderings hold on two different PETSc builds. What hybrid execution saves in MPI it gives back, in part, as threads waiting on each other. Those samples are consistent with the poor 2D 16×8 result, but do not exclude a cache, bandwidth or imbalance mechanism. Section 6.1.4 sets that boundary.

6.1.4 RQ4: How far do the trends generalise, and what would establish a mechanism?

These measurements say what happens. They do not say what is happening in the hardware underneath, and that is the level left open. Knowing that threads wait does not say what they are waiting for. Three causes fit the data equally well: PETSc threads only part of its solve path, the threaded regions may be too short to pay back the cost of a barrier, and memory bandwidth may already be saturated. All three would look the same here. The exposure share has the same problem. It cannot separate message latency, bandwidth, cost that grows with the number of ranks, and imbalance elsewhere in the iteration. The profiled and unprofiled quantities also come from different instruments, so they cannot be added into one time budget. No hardware mechanism is established here.

How far the trends generalise is bounded by the scope set out in Section 6.2: two structured-grid Poisson problems, one PETSc version and compiler environment, one CG+GAMG configuration, solves that reuse a fixed operator, and PETSc’s default row decomposition. Within that scope the patterns hold across two dimensionalities, six problem sizes and two scaling regimes, and the exposure and OpenMP-wait orderings that go with them reproduce on a second PETSc build. The 2D and 3D campaigns are not two independent checks, though. Both use the same row decomposition, and that decomposition is also what makes them differ. Section 7.3 lists what would extend them: hardware counters for the mechanism, a second and third fixed problem size

for the saturation ordering, and node counts beyond 32 to find the turning points this campaign brackets only from below.

6.2 Threats to Validity

The matrix covers two structured-grid Poisson problems, one PETSc version and compiler environment, one CG+GAMG configuration, and solves that reuse a fixed operator. Nothing here need transfer to unstructured matrices, other preconditioners, or solver phases that call dense BLAS. The reuse is the limit that bites hardest: setup was a median 69% of measured time in the single-solve campaign of Section 3.8, which ranked the layouts differently. An application dominated by one-off solves should expect that ranking, not this one.

The largest limit is the decomposition. Both drivers take PETSc’s default contiguous row blocks, so a subdomain is a slab and not a cube, and the halo follows the $\min(L, mn)$ form of Section 3.2 rather than the $N^{2/3}$ of a Cartesian decomposition. At 164.56 million unknowns on 32 nodes that gives flat MPI a halo of $2.03L$ in 3D and $0.64L$ in 2D, against about $0.18L$ and $0.02L$ for cube-shaped subdomains. The layout comparison survives this, because all four layouts run the same code and the model reproduces their measured ordering at all 48 points checked. The explanation does not: hybridisation reduces exposure to a halo that this decomposition makes large, and a DMDA driver would start from a much smaller one. Section 7.3 names the run that would settle how much of the advantage remains.

Three repeats give an observed range, not a significance test. The unknowns-per-core filter allows at most two node counts per problem size, so every strong-scaling step inside the formal matrix covers one doubling, and reading the same points as two weak-scaling chains lengthens the curves without adding measurements. The fixed-size extension supplies genuine six-point curves, but at one problem size per dimension, so whether the 3D saturation ordering holds at other sizes is untested. Its spreads are also larger, up to 11.1%, because the per-solve time at 32 nodes is only tens of milliseconds.

Weak-scaling efficiency is computed against each layout’s own single-node time, so it measures degradation and not speed, and it rewards a layout whose baseline is poor. It must be read together with Figure 4.1 and Figure 4.3, never on its own.

Dividing by the iteration count removes the 9–13 iterations in 2D and 6–7 in 3D, but not the GAMG work that changes with the decomposition inside an iteration. The single-node configuration-space sweep is weaker still. It used the default rank-dependent block-Jacobi solver, and its double-dashed `--ksp_rtol 1e-5` was never consumed by PETSc (Section 3.3), so those runs converged against a mesh-dependent tolerance reaching 2.4×10^{-10} at 6400^2 . It therefore sets the search bound and nothing more. No default-versus-GAMG comparison is drawn from it, and the conclusions that do rest on it hold at a fixed rank count or per iteration, where the tolerance is common to the runs being compared. All formal runs pass the option with a single dash.

The formal runs link threaded LibSci, but Section 5.3 shows the repeated sparse solve barely calls it, at most 2.1% of solve-window core-time, so it is ruled out as an explanation rather than left unproven. MAP timings stay out of the scaling curves because profiler overhead is large and uneven, and for the same reason the profile shares are read as an ordering between layouts and not as absolute costs. There is one profile per cell and none at 128×1 . Hardware-mechanism claims remain outside the evidence established here.

Chapter 7

Conclusion and Future Work

7.1 Conclusions

The corrected 2D and 3D campaigns contain 264 formal runs across 88 configurations from one to 32 ARCHER2 nodes, and the matched fixed-size extension adds 144 more across 48. Sixteen pilot logs are kept for the record but excluded from the statistics. Every formal run does one warm-up solve and 19 timed solves with the GAMG hierarchy reused, and every one was validated against the build, layout, convergence and library PETSc itself reports.

At constant work per core, two to four threads per rank is the reliable range in both dimensions, and eight threads per rank is not competitive in either. Past that point, 2D and 3D behave differently.

In 2D the layout effect grows with the node count. Every hybrid layout improves relative to flat MPI as nodes are added, and does so at every step in four of the six layout-and-chain series. 64×2 is faster per iteration than flat MPI at all 11 size-node points, by 4.3% at one node and 12.8% at 16, and at 164.56M unknowns on 32 nodes it reaches 794.8 million equations per second, 13.1% above flat MPI on raw throughput. 32×4 starts 9.6% slower at one node and ends 6.3% faster at 32.

In 3D the effect is larger but follows no such trend. The best gain in throughput is 33.2%, at 41m on four nodes, and two-thirds of the hybrid measurements are faster per iteration than flat MPI. No series improves at every step, and which layout is best changes with problem size and node count. At 165m on 32 nodes 64×2 reaches 510.4 million equations per second, 30.1% above flat MPI on raw throughput. The same 165m problem on 16 nodes, where each core holds twice as much of it, is the one point in the 3D campaign at which flat MPI has the highest throughput of the four.

Held at a fixed problem size the same layouts spread much further apart. The 2D curves stay within a factor of 1.81 of each other from one node to 32, and all four are still getting faster at 4096 cores. The 3D curves come apart. Three of the four stop improving and start getting slower, and the node count at which that happens rises with their thread count: eight nodes for flat MPI, 16 for 64×2 and 32×4 . Only 16×8 is still improving at 32. Flat MPI ends $2.6\times$ slower there than at its own best point, while 16×8 , which was last in every weak-scaling comparison, is $3.81\times$ faster than flat MPI at that same point. What separates the layouts at a fixed size is where each one stops scaling, so any recommendation must name the scaling regime it was measured in.

Profiling measures both halves of the trade: what hybrid execution saves in communication, and what it pays for in threading. Hybrid layouts spend less time exposed to halo exchange and reductions, while eleven MAP profiles show the share of core time spent waiting in the OpenMP runtime rising from 48.3% to 76.5% with threads per rank at eight nodes, on two builds. BLAS accounts for at most 2.1% of core time during the solve. These observations show where the time goes in software, not what causes it in the hardware.

These measurements replace the invalid launcher-era layout rankings, and Section 7.2 turns them into recommendations. All of it is measured on PETSc’s default row decomposition, which Section 6.2 shows makes the halo large, so how much of the advantage survives a DMDA driver is untested. The evidence establishes these performance patterns and names the software cost behind them, but not the hardware mechanism underneath.

7.2 Practical Guidance for PETSc on ARCHER2

In 2D, default to 64×2 . It is faster per iteration than flat MPI at every measured point, by 4.3% at one node and 12.8% at 16, so the larger the job the safer the choice. Two things are still worth a test. Flat MPI wins on small one-node jobs, with 5.9% and 9.5% higher raw throughput at 5m and 10m, because it converges in one fewer GAMG iteration. 32×4 is worth trying above roughly eight nodes, where it turns from slower than flat MPI to faster.

In 3D, test 32×4 and 64×2 at the node count you will actually use, and do not extrapolate from a smaller one. 32×4 wins more points and gives the largest single margin, 33.2% at 41m on four nodes, but 64×2 is better at 165m on 32 nodes, where it beats flat MPI by 30.1%. At 16 nodes on the $\sim 80k$ chain no hybrid layout wins at all.

Do not default to eight threads per rank. 16×8 wins no raw-throughput point in either dimension at constant work per core, and its 2D weak-scaling efficiency is not evidence for it (Section 6.2). Try it in one case only: a fixed 3D problem pushed past the point where the other layouts stop scaling. At about 4.9 thousand unknowns per core on 32 nodes it was the fastest layout measured and the only one of the four still improving.

Underneath all three, choose for the regime the job actually runs in. Above roughly 25 thousand unknowns per core the layouts differ by single-digit percentages, and 64×2 in 2D or 32×4 in 3D is a safe default. If instead the problem is fixed and the node count is being pushed for time to solution, the choice is worth factors, and the most-threaded layouts are the ones that keep scaling. A benchmark run at one node, or at comfortable work per core, does not answer that second case.

7.3 Future Work

The MAP profiles show that the threads wait. Hardware counters are needed to show what they are waiting for. Memory-bandwidth and cache-miss counters, taken per layout at a fixed size and node count, would separate the three candidate causes that the profiles cannot tell apart: PETSc threading only part of the solve path, threaded regions too short to pay back the cost of an OpenMP barrier, and bandwidth saturating before all cores are busy. Measuring PETSc’s threaded regions directly, rather than sampling them, would settle the first of those without a counter at all.

The profiled matrix is also thin: one problem size, two node counts on the production build, three hybrid layouts, and one run per cell. It has no flat-MPI cell to anchor the trend, because MAP cannot launch that layout. Repeating it at a second problem size, and at the node counts where the 3D fixed-size curves stop improving, would show whether the OpenMP wait share predicts where a layout stops scaling or merely rises alongside it.

The next thing to change is the decomposition itself. Rebuilding both drivers on DMDA, so that a subdomain is a cube rather than a slab, and repeating the same four layouts would cut the 3D halo of Section 6.2 by about an order of magnitude. That would show how much of the hybrid advantage comes from threading and how much from PETSc’s default row decomposition. Nothing else in the setup would need to change, so the two campaigns would be comparable point for point.

Two extensions follow from the fixed-size results. A second and third fixed problem size would show whether the order in which the 3D layouts stop scaling is a property of the layouts or

of 20M unknowns. Node counts beyond 32 would show whether 32×4 keeps getting slower, and would locate the point at which 16×8 stops improving, which this campaign never reached. More repeats where two layouts finish close together would support confidence intervals in place of min–max ranges. Further experiments should include preconditioner setup, changing operators, unstructured matrices, other compiler environments, and GPU execution.

Bibliography

- [1] Satish Balay et al. *PETSc/TAO Users Manual, Revision 3.24*. Tech. rep. ANL-21/39 Rev. 3.24. Argonne National Laboratory, 2025. DOI: 10.2172/2998643.
- [2] Satish Balay et al. “Efficient Management of Parallelism in Object-Oriented Numerical Software Libraries”. In: *Modern Software Tools in Scientific Computing*. Ed. by Eirik Arge, Are Magnus Bruaset, and Hans Petter Langtangen. Boston: Birkhäuser, 1997, pp. 163–202. DOI: 10.1007/978-1-4612-1986-6_8.
- [3] PETSc Development Team. *Changes: 3.22*. 2024. URL: <https://petsc.org/release/changes/322/> (visited on 07/22/2026).
- [4] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*. Nov. 2023. URL: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf> (visited on 07/22/2026).
- [5] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. Nov. 2021. URL: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf> (visited on 07/22/2026).
- [6] Holly Judge. “Effective Use of MPI+OpenMP on a Cray EX Supercomputer”. In: *Proceedings of the Cray User Group 2022*. 2022. URL: https://cug.org/proceedings/cug2022_proceedings/includes/files/pap104s2-file1.pdf (visited on 07/22/2026).
- [7] Michael Lange et al. “Achieving Efficient Strong Scaling with PETSc Using Hybrid MPI/OpenMP Optimisation”. In: *Euro-Par 2012: Parallel Processing Workshops*. Ed. by Julian M. Kunkel, Thomas Ludwig, and Hans Werner Meuer. Berlin, Heidelberg: Springer, 2013, pp. 11–20. DOI: 10.1007/978-3-642-38750-0_8.
- [8] ARCHER2 Service. *ARCHER2 Hardware*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/hardware/> (visited on 07/22/2026).
- [9] ARCHER2 Service. *Performance Tuning*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/tuning/> (visited on 07/22/2026).
- [10] Magnus R. Hestenes and Eduard Stiefel. “Methods of Conjugate Gradients for Solving Linear Systems”. In: *Journal of Research of the National Bureau of Standards* 49.6 (1952), pp. 409–436. DOI: 10.6028/jres.049.044.
- [11] Klaus Stüben. “A Review of Algebraic Multigrid”. In: *Journal of Computational and Applied Mathematics* 128.1–2 (2001), pp. 281–309. DOI: 10.1016/S0377-0427(00)00516-1.
- [12] Linaro Limited. *Linaro MAP*. Linaro Forge 25.0.4 documentation. 2025. URL: https://docs.linaroforge.com/25.0.4/html/forge/forge/introduction_to_forge/map.html (visited on 07/22/2026).
- [13] PETSc Development Team. *KSP Tutorial ex2.c: Solves a Linear System in Parallel with KSP*. PETSc source tree, `src/ksp/ksp/tutorials/ex2.c`. 2025. URL: <https://petsc.org/release/src/ksp/ksp/tutorials/ex2.c.html> (visited on 08/08/2026).

Appendix A

Reproducibility: Repository and Scripts

The repository accompanying this dissertation contains the benchmark sources, the ARCHER2 job scripts, every raw log, and the analysis pipeline that turns one into the other.

A.1 Layout

Table A.1 gives the directory layout of the repository.

Table A.1: Repository layout. Raw logs and retrieved job CSVs are immutable evidence. Everything under a *derived* heading is regenerated by the scripts.

Path	Contents
<code>src/</code>	Benchmark drivers (Section 3.2)
<code>scripts/ksp/{2d,3d}/lib/</code>	Slurm job scripts and problem-size tables
<code>scripts/petsc_configure_archer2.sh</code>	PETSc configure recipes for both builds
<code>analysis/data/<campaign>/logs/</code>	Raw <code>-log_view</code> output, immutable
<code>analysis/data/<campaign>/csv/source/</code>	Retrieved job CSVs, byte-for-byte
<code>analysis/maps/<campaign>/</code>	Linaro MAP profiles, immutable
<code>analysis/data/<campaign>/csv/derived/</code>	Derived: normalised and aggregated CSVs
<code>analysis/data/<campaign>/tables/</code>	Derived: reference tables
<code>analysis/plots/<campaign>/</code>	Derived: figures, as PNG and PDF pairs
<code>analysis/scripts/<campaign>/</code>	Analysis code and its unit tests
<code>docs/dissertation_latex/</code>	This document and <code>sync_figures.sh</code>

The campaigns are named for the study that produced them. The results in this dissertation come from `nproc_2d_libsci` and `nproc_3d_libsci`, which hold both the validated repeated-solve matrices and, under `strong_scaling_20m`, the fixed-size extension of Section 4.1.2. The same 2D campaign also holds the earlier `multi_node`, `single_node_hybrid` and `single_node_pure_mpi` run sets, collected on the same build and driver while the configuration space was being fixed. Of these only `multi_node` is used in the text, for Table 3.3. `size_grid` holds the single-node configuration-space sweep of Section 3.5, and `profiling_2d_map_baseline` the baseline MAP profiles of Section 5.3. The earlier `ksp_2d_scaling`, `ksp_3d_scaling`, `mpi_only` and `fixed128_multinode` campaigns are retained as provenance and are not used for any layout claim, for the reasons given in Section 3.3 and Section 6.2.

A.2 Evidence Index

Every numbered figure and table in this dissertation is derived from raw job output by one of six scripts. Table A.2 gives the chain for each: which script produced it, which derived file that script writes, and which raw evidence the script consumed. To check any number, run the script named in its row and compare. To check the number’s provenance instead, read the raw evidence directly, which is retained unmodified.

Table 2.1 and Table 4.2 state definitions rather than measurements and have no evidence chain: the first classifies the execution models and the second restates the admission rule of Section 3.4. Figure 2.1 is reproduced from the ARCHER2 hardware documentation. Figure 3.1 is a schematic and plots no measurement; it is drawn by `analysis/scripts/cross_dimension/plot_decomposition_schematic.py` and shows only the geometry that Section 3.2 states in symbols.

Table A.2: Evidence index. Script paths are relative to `analysis/scripts/`, derived and raw paths to `analysis/`. Log counts are the files present, which for the 3D fixed-size campaign exceeds the 72 accepted runs because superseded job attempts are retained. The last two rows are read directly from a named file and have no derived stage.

Figures and tables	Generated by	Derived output	Raw evidence
Table 3.1, Table 3.2, Figure 3.2	<code>size_grid/ analyze_single_node_ sweep.py</code>	<code>data/size_grid/tables/ iterations_vs_ layout.csv, fullnode_cost_per_ iteration.csv</code>	<code>data/size_grid/logs/ 213 logs</code>
Table 3.3	<code>nproc_2d_libsci/ analyze_multinode_ thread_limit.py</code>	<code>tables/multinode_ thread_limit.csv</code>	<code>logs/multi_node/ 45 logs, reconciled against csv/derived/fullnode_ runs.csv</code>
Table 4.1, Table 4.4, Figure 4.1, Figure 4.2, Figure 4.3, Figure 4.4, Figure B.1, Figure B.2	<code>nproc_{2d,3d}_libsci/ analyze_repeated_ fullnode_logs.py</code>	<code>csv/derived/repeated_ fullnode_formal_ summary.csv</code>	<code>logs/repeated_ fullnode/ 140 logs per dimension</code>
Table 4.5, Table 4.6, Figure 4.5– Figure 4.7, Table B.1, Table B.2	<code>strong_scaling_20m/ analyze_strong_ scaling_20m.py, then emit_runtime_ grid.py for the two grids</code>	<code>csv/derived/strong_ scaling_20m/*.csv, tables/strong_ scaling_20m/*.csv</code>	<code>logs/strong_ scaling_20m/ 72 (2D) and 108 (3D) logs</code>
Table 4.3	<code>cross_dimension/ summarize_best_ layouts.py</code>	<code>tables/weak_scaling_ best_layouts.csv</code>	the two formal summaries above
Table 4.7	<code>cross_dimension/ summarize_cross_ dimension.py</code>	<code>tables/cross_dimension_ summary.csv</code>	joins the three rows above
Table 5.2, Figure 5.1	<code>map_profiles/ analyze_map_ profiles.py</code>	<code>csv/derived/map_ profiles/map_call_tree_ shares.csv</code>	<code>maps/ 12 MAP profiles, 11 usable</code>
Table 5.4, Figure 5.2– Figure 5.5	<code>nproc_2d_libsci/ analyze_event_ breakdown.py (covers both dimensions)</code>	<code>tables/repeated_ fullnode_event_ matrix.csv, repeated_fullnode_ exposure_fits.csv</code>	<code>logs/repeated_ fullnode/ as above</code>
Table 2.2, Table 3.4	read directly	—	<code>scripts/ksp/ diagnostics/ petsc_diagnostics. 14598711.out</code>
Table 2.3, Table 5.1	read directly	—	one named <code>-log_view</code> log each, cited in place

A.3 Regenerating the results

From the repository root, after creating the analysis environment with `python3 -m venv analysis/.venv` and `analysis/.venv/bin/pip install -r requirements.txt`:

```
# unit tests (30)
analysis/.venv/bin/python -m pytest -c analysis/pytest.ini analysis/scripts

# single-node configuration-space sweep -> Table 3.1, Table 3.2, Figure 3.1
analysis/.venv/bin/python \
    analysis/scripts/size_grid/analyze_single_node_sweep.py

# multi-node threads-per-rank restriction -> Table 3.3
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_multinode_thread_limit.py

# 2D and 3D repeated-solve campaigns -> Chapter 4 weak-scaling figures
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_repeated_fullnode_logs.py
analysis/.venv/bin/python \
    analysis/scripts/nproc_3d_libsci/analyze_repeated_fullnode_logs.py

# matched fixed-20m strong scaling -> Chapter 4 strong-scaling figures
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/analyze_strong_scaling_20m.py \
    --dimension 2d
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/analyze_strong_scaling_20m.py \
    --dimension 3d

# full layout-by-node runtime grids -> Table B.1, Table B.2
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/emit_runtime_grid.py

# duplicate-batch reproducibility check -> Section 4.1.2
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/compare_superseded_batches.py

# event-level breakdown and exposure fits -> Section 5.4
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_event_breakdown.py

# trade-off figures, panelled by chain -> Figure 5.4, Figure 5.5
# reads only the derived table above, so it needs no raw logs
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/plot_exposure_tradeoff_by_chain.py

# Linaro MAP call-tree shares -> Section 5.3
analysis/.venv/bin/python \
    analysis/scripts/map_profiles/analyze_map_profiles.py

# weak-scaling best layout per point -> Table 4.3
```

```
analysis/.venv/bin/python \  
    analysis/scripts/cross_dimension/summarize_best_layouts.py  
  
# cross-dimensional comparison          -> Table 4.7  
analysis/.venv/bin/python \  
    analysis/scripts/cross_dimension/summarize_cross_dimension.py  
  
# copy the generated PDFs into this document's figures/ directory  
bash docs/dissertation_latex/sync_figures.sh
```

Each script prints the validated logs and aggregated configurations, and exits non-zero on a failed check. Build the document from its directory with:

```
cd docs/dissertation_latex  
biber draft  
latexmk -pdf -interaction=nonstopmode -halt-on-error draft.tex
```

Appendix B

Supplementary Figures and Tables

B.1 Weak-scaling throughput

Figure B.1 and Figure B.2 give raw repeated-solve throughput for the weak-scaling campaign of Section 4.2. Throughput rises almost linearly with the node count along both chains in both dimensions, which is the result they carry. They cannot carry the layout comparison: on a logarithmic axis the four layouts differ by a few per cent against a range of three orders of magnitude, which is why Figure 4.1 and Figure 4.3 normalise each layout against flat MPI at the same size–node pair instead.

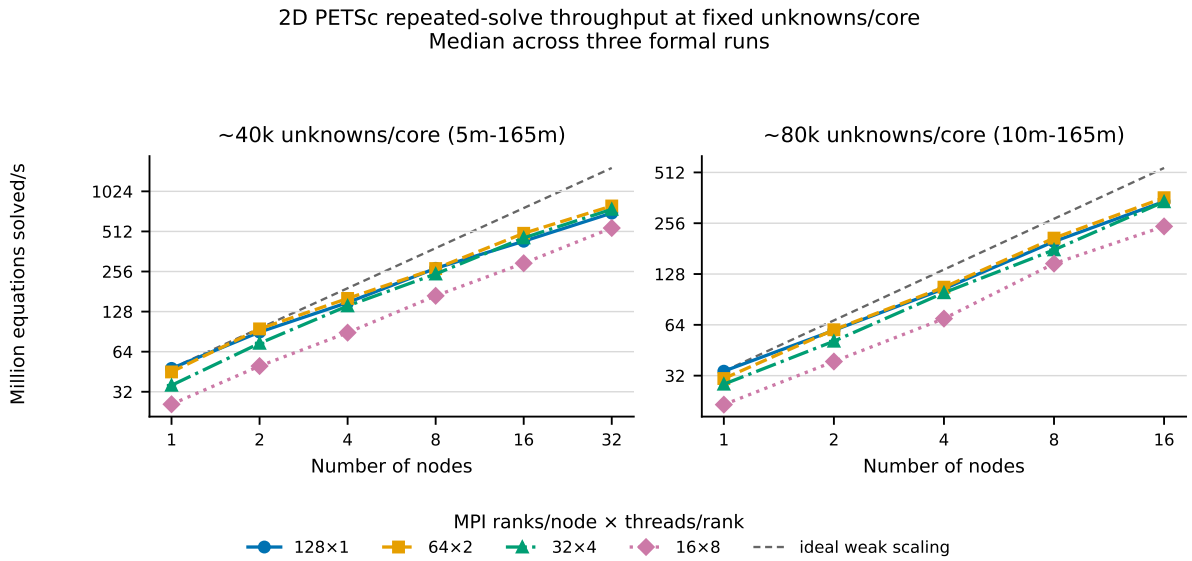


Figure B.1: Median 2D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs. All layouts use 128 cores per node, so a fixed node count means a fixed core count.

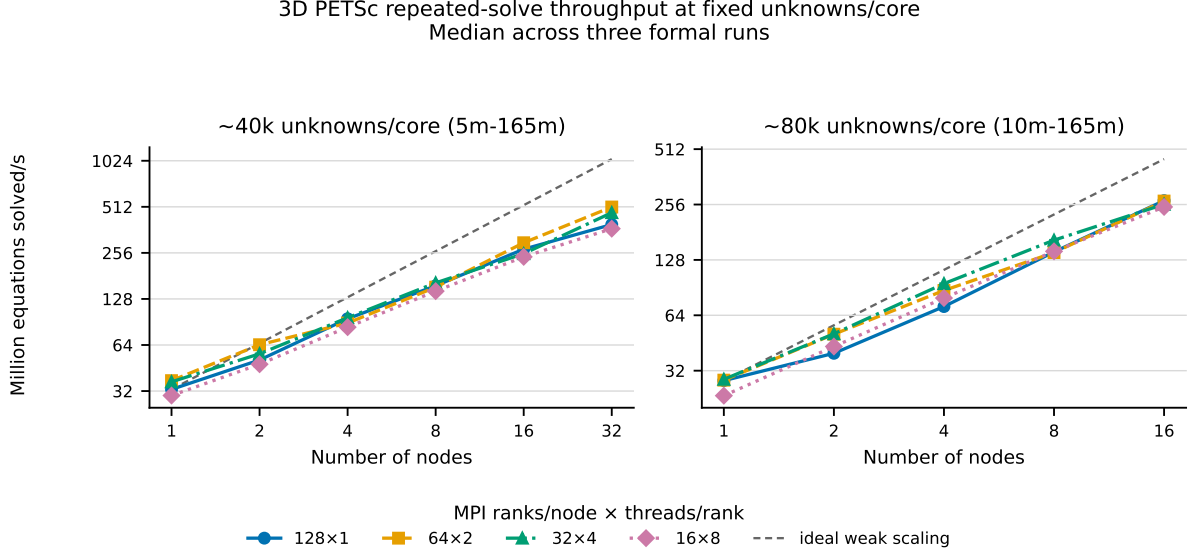


Figure B.2: Median 3D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs.

B.2 Fixed-size runtime grids

Section 4.1.2 presented these two campaigns as curves. Figure 4.5 showed the four 2D layouts falling together across the whole range and still falling at 32 nodes, and Figure 4.6 showed the 3D layouts separating instead, as three of the four reached a minimum and turned upward.

Table B.1 and Table B.2 give every point behind those two figures. Chapter 4 tabulates only the fastest layout at each node count and the 32-node endpoint, so these grids are what an examiner needs to check a runtime read off either figure. The last two columns state what the figures show as a shape: the speedup each layout gains over the final doubling, and the largest three-repeat spread it showed at any node count on the curve.

The 2D grid falls monotonically in all four rows. No layout has reached its minimum within 32 nodes, but the final doubling returns 1.11 to 1.39 $\times$ rather than the 2 $\times$ of the earlier steps, so the curves are flattening even though none has turned. The 3D grid turns upward in three rows out of four, at eight nodes for 128 $\times$ 1 and 16 nodes for 64 $\times$ 2 and 32 $\times$ 4. The last-doubling column is below 1.00 for exactly those layouts that are getting slower.

Table B.1: 2D fixed-size campaign, 4500² and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row. All layouts fill the node, so 32 nodes is 4096 cores in every row. Iteration counts range from 9 to 11.

Layout	Median s/solve at N nodes						16 $\rightarrow$ 32	Spread
	1	2	4	8	16	32		
128 $\times$ 1	0.7704	0.3399	0.1351	0.0558	0.0313	0.0282	1.11	7.9%
64 $\times$ 2	0.8260	0.3365	0.1265	0.0541	0.0348	0.0277	1.26	2.7%
32 $\times$ 4	0.8839	0.3944	0.1465	0.0632	0.0388	0.0317	1.22	10.8%
16 $\times$ 8	1.1325	0.5223	0.2272	0.0973	0.0568	0.0409	1.39	4.8%

Table B.2: 3D fixed-size campaign, 273^3 and 20.35M unknowns, median seconds per solve over three runs at every layout and node count. Columns as in Table B.1. A 16-to-32-node figure below 1.00 means the layout is slower at 32 nodes than at 16. Iteration counts are 6 or 7 throughout.

Layout	Median s/solve at N nodes						16→32	Spread
	1	2	4	8	16	32		
128×1	0.9066	0.5149	0.2208	0.0997	0.1776	0.2599	0.68	4.2%
64×2	0.8288	0.4041	0.2342	0.0936	0.0748	0.1703	0.44	3.2%
32×4	0.8022	0.4028	0.2145	0.1192	0.0652	0.0747	0.87	11.1%
16×8	0.9707	0.4718	0.2475	0.1330	0.1017	0.0681	1.49	7.6%

Appendix C

Full log_view Excerpts